

DOCUMENTO DE EJECUCIÓN

PLANIFY

Versión 4.0

Barrera Gutierrez Amara
Becerra Rodriguez Paul Ignacio
Cayo Villena Daniel Nicolás
Corrales Peña Renato Oscar
Mazza Córdova Káel Alexander

Versión	Fecha	Integrantes
Versión 1.0	05/09/2025	Barrera Gutierrez Amara Becerra Rodriguez Paul Ignacio Cayo Villena Daniel Nicolás Corrales Peña Renato Oscar Mendoza Sueros Marela
Versión 2.0	13/10/2025	Barrera Gutierrez Amara Becerra Rodriguez Paul Ignacio Cayo Villena Daniel Nicolás Corrales Peña Renato Oscar Mazza Córdova Káel Alexander Mendoza Sueros Marela
Versión 3.0	07/11/2025	Barrera Gutierrez Amara Becerra Rodriguez Paul Ignacio Cayo Villena Daniel Nicolás Corrales Peña Renato Oscar Mazza Córdova Káel Alexander Mendoza Sueros Marela
Versión 4.0	26/11/2025	Barrera Gutierrez Amara Becerra Rodriguez Paul Ignacio Cayo Villena Daniel Nicolás Corrales Peña Renato Oscar Mazza Córdova Káel Alexander

Índice

1. Introducción a Planify.....	4
2. Sección Administrativa.....	6
a. Normas de codificación.....	6
i. Codificación de Requisitos Funcionales (RF).....	6
ii. Codificación de Requisitos No Funcionales (RNF).....	7
iii. Codificación de Actores del Sistema (AC).....	7
iv. Codificación de Casos de Uso (CU).....	8
v. Codificación de Interfaces de Usuario (Mk).....	8
1. Pantallas / Interfaces.....	8
2. Nomenclatura de Archivos en GitHub.....	9
vii. Codificación de Gestores del Sistema (G).....	9
vii. Codificación de Entidades del Modelo (ENTM).....	10
ix. Codificación de Entidades Relacionales (ENTR).....	10
x. Codificación de Funcionalidades o Métodos (FP).....	11
xi. Codificación de Componentes del Sistema (COMP).....	11
xii. Codificación de Casos de Prueba (CD).....	12
b. Matriz de Trazabilidad.....	12
3. Análisis Planify.....	14
a. Requerimientos.....	14
i. Requerimientos de Usuario.....	15
ii. Requerimientos Funcionales.....	19
iii. Requerimientos No Funcionales.....	23
b. Modelo de Requisitos.....	24
i. Modelo de casos de uso.....	24
Acceso al diagrama de casos de uso.....	25
Especificaciones de caso de uso.....	26
ii. Modelo de presentación.....	27
Acceso al Diagrama de Navegabilidad.....	29
iii. Información.....	29
Acceso al diagrama de Clases de Análisis.....	30
c. Modelo Entidad Relación.....	34
Acceso al modelo entidad relación.....	34
1. Diccionario de Diagrama entidad relación.....	34
4. Diseño.....	43
a. Diagramas de Secuencia.....	44
Acceso a los diagramas de secuencia.....	46
b. Diagrama de Clases de Diseño.....	48
i. Diccionario del Diagrama de Clases de Diseño.....	50
c. Modelo Relacional.....	52
i. Algoritmo de mapeo de 7 pasos.....	52

ii. Diccionario Modelo Relacional.....	55
Acceso al Diccionario Diagrama del Modelo Relacional.....	57
d. Arquitecturas.....	57
i. Diagrama de Componentes.....	58
ii. Diagrama de Despliegue.....	61
5. Implementación.....	63
a. Aplicativo.....	63
1. Registrarse (MK-002 y CU-01).....	64
2. Recuperar contraseña (MK-017 y CU-03).....	67
3. Crear Concepto (MK-020 y CU-07).....	69
4. Editar Concepto (MK-020 y CU-07).....	70
5. Crear Meta (MK-018 y CU-04).....	71
6. Editar Cuenta (MK-011 y CU-11).....	73
7. Consultar Progreso Meta (MK-018 y CU-06).....	75
8. Registrar movimiento(MK-019 y CU-07).....	76
9. Consultar Balance (MK-009 y CU-08).....	77
10. Generar Reporte (MK-010 y CU-09).....	78
11. Iniciar sesión (MK-001 y CU-02).....	79
b. Modelo Físico.....	80
Acceso al Modelo Físico.....	81
6. Pruebas.....	83
a. Acceso al Plan de Pruebas del Software Planify.....	84
7. Despliegue Planify.....	86
a. Manual de Usuario.....	87
b. Plan de entrega.....	87
8. Mantenimiento.....	92
a. Plan de mantenimiento.....	92
9. Referencias.....	93

Introducción

1. Introducción a Planify

Planify es una aplicación orientada a la gestión financiera personal, cuyo propósito es ayudar a los usuarios a organizar sus ingresos, egresos y metas económicas mediante un conjunto de funciones estructuradas. El sistema se concibe como una herramienta digital que no solo permite registrar movimientos de dinero, sino también visualizar su impacto en balances y reportes, con un enfoque en la planificación financiera y el control de gastos.

Objetivo principal

El objetivo de Planify es facilitar la toma de decisiones financieras al usuario, ofreciéndole:

- Un control claro de sus ingresos y egresos.
- La capacidad de definir y dar seguimiento a metas financieras.
- La generación de reportes detallados que consoliden la información en periodos definidos.

De esta forma, el sistema busca brindar una visión clara y ordenada de la situación económica del usuario y presenta las siguientes funcionalidades.

1. Autenticación y gestión de cuentas

- o Registro de usuario.
- o Inicio de sesión.
- o Recuperación y cambio de contraseña.
- o Eliminación de cuenta.

2. Gestión de movimientos

- o Registro de ingresos, egresos y gastos espontáneos.
- o Registro de conceptos (categorías de movimientos).

3. Gestión de metas

- o Crear, editar y consultar metas financieras.
- o Monitorear el progreso mediante indicadores o gráficos.

4. Balance y reportes

- o Consultar balance por periodos y filtros.
- o Generar reportes detallados de ingresos y gastos.
- o Exportar reportes en formatos estándar como PDF o CSV.

Sección Administrativa

2. Sección Administrativa

a. Normas de codificación

Con el fin de mantener la coherencia, trazabilidad y claridad en toda la documentación y desarrollo del sistema, se establecen normas de codificación que permiten identificar y clasificar de manera estructurada los distintos elementos del proyecto. Estas convenciones facilitan la comunicación entre los miembros del equipo, aseguran la uniformidad en la documentación técnica, y permiten una mejor gestión del sistema durante su evolución. A continuación, se detallan los criterios definidos para la identificación.

i. Codificación de Requisitos Funcionales (RF)

Formato:

- RF – XX Nombre del Requisito Funcional

Descripción:

Se utiliza el prefijo RF (Requisito Funcional) seguido de un identificador XX con números.

Este código se empleará en casos de uso, diagramas y documentación técnica, asegurando trazabilidad y consistencia.

Ejemplo:

- RF-01 Registrar Usuario
- RF-02 Generar Reporte de Actividades

ii. Codificación de Requisitos No Funcionales (RNF)

Formato:

- RNF – XX Nombre del Requisito No Funcional

Descripción:

Código con prefijo RNF (Requisito No Funcional), seguido de un identificador XX en números.

El nombre expresa un atributo de calidad: rendimiento, seguridad, usabilidad, disponibilidad, mantenibilidad, entre otros.

Ejemplo:

- RNF-01 Tiempo de Respuesta
- RNF-02 Disponibilidad del 99.9%

iii. Codificación de Actores del Sistema (AC)

Formato:

- AC – XX Nombre del Actor

Descripción:

Código con prefijo AC (Actor), seguido de un identificador con

letras. Cuando un actor principal tiene subtipos, se usa una letra minúscula adicional.

Ejemplo:

- AC-A Usuario
- AC-Aa NuevoUsuario (subtipo de Usuario)
- AC-B Administrador

iv. Codificación de Casos de Uso (CU)

Formato:

- CU – XX Nombre del Caso de Uso

Descripción:

Código con prefijo CU (Caso de Uso), seguido de un identificador con números.

Se usa para documentar las interacciones entre actor y sistema en los diagramas y la documentación.

Ejemplo:

- CU-01 Iniciar Sesión
- CU-02 Editar Perfil de Usuario

v. Codificación de Interfaces de Usuario (Mk)

1. Pantallas / Interfaces

Formato:

- Mk – XXX Nombre de la Interfaz
(*opcional*) sufijos de estado/variante: a, b, c...
(*opcional*) versión: vM.m.p

Descripción:

Código para identificar pantallas o vistas diseñadas en Draw.io y documentadas en GitHub.

- Mk = Interfaz de Usuario.
- XXX = identificador con números.
- Variante = letra minúscula cuando la IU tiene un estado alterno (error, vacío, modal).

Ejemplos:

- Mk-01 Pantalla de Inicio de Sesión v1.0 (RF-01, CU-A, AC-A)
- Mk-02 Tablero de Proyectos v2.1 (RF-01, CU-01, AC-01, AC-02)
- Mk-02a Tablero de Proyectos – Estado Vacío v2.1

2. Nomenclatura de Archivos en GitHub

- Flujos (Draw.io): FL-XX_nombre-vM.m.p.drawio
- Mensajes (documento):
/ui/messages/MSG-XX_nombre.md

vi. Codificación de Objetos del Sistema

Formato:

- O – XX Nombre del Objeto

Descripción:

El prefijo O (Objeto), seguido de un identificador XX en números o letras, identifica los objetos de análisis que modelan una acción concreta dentro de un flujo.

Se utilizan en diagramas de comunicación o secuencia para representar la interacción entre la interfaz (Mk) y el gestor (G).

Ejemplos:

- O-01 :Login
- O-02 :SignUp
- O-03 :Forgot

vii. Codificación de Gestores del Sistema (G)

Formato:

- G – XX Nombre del Gestor

Descripción:

El prefijo G (Gestor), seguido de un identificador XX en números o letras, representa un componente de control dentro de la arquitectura de análisis (equivalente al estereotipo *control* en UML).

Los gestores encapsulan la lógica de negocio y coordinan la comunicación entre interfaces (Mk) y objetos (O), manteniendo la separación de responsabilidades.

Ejemplos:

- G-01 GestorAutenticacion
- G-02 GestorRegistro
- G-03 GestorPassword

vii. Codificación de Entidades del Modelo (ENTM)

Formato:

- ENTM – XX NombreEntidad

Descripción:

El prefijo **ENTM** (Entidad del Modelo), seguido de un identificador **XX** en números o letras, representa una entidad principal dentro del modelo de datos.

Cada entidad describe un conjunto de objetos del mundo real o lógico que comparten las mismas propiedades y comportamientos relevantes para el sistema.

Estas entidades constituyen la base para el diseño de las tablas en la base de datos y permiten establecer relaciones entre los diferentes elementos del dominio.

Ejemplos:

- ENTM-01 User
- ENTM-02 Account
- ENTM-03 Goals

ix. Codificación de Entidades Relacionales (ENTR)

Formato:

- ENTR – XX NombreEntidadRelacional

Descripción:

El prefijo **ENTR** (Entidad Relacional), seguido de un identificador **XX**, se utiliza para representar las entidades que intervienen directamente en el modelo relacional derivado del modelo entidad-relación conceptual.

Estas entidades corresponden a las tablas físicas del sistema en la base de datos y mantienen la estructura necesaria para almacenar la información de manera persistente.

Cada entidad relacional incluye sus respectivos atributos, claves primarias y foráneas, garantizando la integridad referencial y la coherencia de los datos.

Ejemplos:

- ENTR-01 User
- ENTR-02 Account

- ENTR-03 Goals

x. Codificación de Funcionalidades o Métodos (FP)

Formato:

- FP – XX NombreFuncionalidad

Descripción:

El prefijo FP (Functional Procedure), seguido de un identificador XX, representa las funcionalidades o métodos implementados en el sistema, tanto en la capa de presentación como en la de negocio.

Cada método define una acción específica que puede ser ejecutada por un componente, gestor o interfaz, ya sea para procesar información, validar datos o renderizar elementos visuales.

Esta codificación facilita la documentación del comportamiento dinámico del sistema y su trazabilidad dentro de los diferentes módulos.

Ejemplos:

- FP-01 isSignUp
- FP-02 renderLoginForm
- FP-03 signUp

xi. Codificación de Componentes del Sistema (COMP)

Formato:

- COMP – XX NombreComponente

Descripción:

El prefijo COMP (Componente), seguido de un identificador XX, identifica los módulos o unidades funcionales que agrupan un conjunto de funcionalidades relacionadas dentro del sistema. Cada componente cumple un rol específico y encapsula tanto la lógica como las interfaces necesarias para su operación, contribuyendo a la modularidad y mantenibilidad del software.

Esta codificación permite organizar la arquitectura del sistema en bloques coherentes y reutilizables.

Ejemplos:

- COMP-01 GestiónUsuarios
- COMP-02 GestiónMetas

xii. Codificación de Casos de Prueba (CD)

Formato:

- CD–TipoDePrueba–Número

Descripción:

El prefijo CD (Caso de Prueba) identifica cada validación diseñada dentro del plan de pruebas de Planify.

El segundo componente, TipoDePrueba, especifica la naturaleza del caso (por ejemplo, Validación, Integración, Interfaz, Funcional, etc.), y el número secuencial (001, 002, 003...) permite mantener un control ordenado y trazable.

Cada caso de prueba CD está vinculado a uno o más requisitos funcionales (RF-XX) y gestores (G-XX), garantizando coherencia entre el diseño, la ejecución de pruebas y la arquitectura de análisis del sistema.

Ejemplos:

- CD-INT-004 Integración Registro de Movimientos
- CD-UN-001 Verificación de la longitud de contraseña inválidas
- CD-SIS-005 Sistema Creación Exitosa de una Nueva Meta de Ahorro
- CD-VAL-007 Validación verificación del formato de correo electrónico

b. Matriz de Trazabilidad

La Matriz de Trazabilidad de Requisitos para el software Planify. Este documento fundamental vincula cada requisito (funcional y no funcional) con todos los artefactos de desarrollo, garantizando que cada requisito sea implementado y verificado, asegurando la cobertura total del proyecto. (véase Figura 1 en la siguiente hoja de forma horizontal).

Figura 1
Vista General de Matriz de Trazabilidad

Código de Requisito	Requisitos		Casos de Uso	Mockups	Diagrama de Clases de Análisis	Diagrama de Secuencia	Diagrama de Clases	Artefactos			Métodos	Componentes	Casos de Prueba				Despliegue
	Descripción de requisito	Tipo						Modelo Entidad-Relación	Modelo Relacional	Modelo Físico			Pruebas Unitarias	Pruebas de Integración	Pruebas de Validación	Pruebas de Sistema	
RF-01	El sistema permitirá a los usuarios iniciar sesión utilizando su nombre de usuario y contraseña.	Funcional	CU-02 Iniciar sesión	MK-001 Login	DCA-001 Login y SignUp	SD-004 Iniciar Sesión	DC-001 Planify	ENTM-01 User	ENTR-01 User	FP-01 isSignUp FP-02 renderLoginForm	COMP-01 GestiónUsuarios	(Pendiente)	CD-INT-03	CD-VA-08	(Pendiente)	(Pendiente)	
RF-02	El sistema permitirá a los usuarios registrarse mediante su correo electrónico, nombre de usuario y contraseña.	Funcional	CU-01 Registrarse	MK-002 Sign Up	DCA-001 Login y SignUp	SD-003 Registrarse		ENTM-01 User	ENTR-01 User	FP-03 signUp FP-04 confirmPassword FP-05	COMP-01 GestiónUsuarios		CD-UN-01 CD-UN-02	CD-INT-03	CD-VA-07		(Pendiente)
RF-03	El sistema permitirá a los usuarios recuperar su contraseña enviando un enlace al correo electrónico registrado.	Funcional	CU-03 Recuperar contraseña	MK-010 Forgot Password	DCA-001 Login y SignUp	SD-005 Recuperar Contraseña		ENTM-01 User	ENTR-01 User	FP-11 handleForgotPassword FP-12 handleVerifyOp FP-13 handleResetPassword	COMP-01 GestiónUsuarios	(Pendiente)	(Pendiente)	(Pendiente)	(Pendiente)		CD-SIS06
RF-04	El sistema permitirá a los usuarios seleccionar un rango de fechas y visualizar el balance de ingresos y gastos dentro de ese período.	Funcional	CU-08 Consultar balance	MK-005 Balance	DCA-002 Balance y Daily Input	SD-001 Consultar Balance		ENTM-02 Account ENTM-05 Movement	ENTR-02 Account ENTR-05 Movement	(Pendiente)	(Pendiente)	(Pendiente)	(Pendiente)	(Pendiente)	(Pendiente)		(Pendiente)
RF-05	Funcionalidad para registrar ingresos y gastos por categorías definidas por el usuario.	Funcional	CU-07 Registrar movimiento (general)	MK-009 Daily Input	DCA-002 Balance y Daily Input	SD-007 Registrar Movimiento		ENTM-05 Movement	ENTR-05 Movement	(Pendiente)	(Pendiente)	(Pendiente)	CD-INT-04	(Pendiente)	(Pendiente)		(Pendiente)
RF-06	El sistema permitirá registrar gastos e ingresos no planificados fuera de las categorías preestablecidas.	Funcional	CU-07.2 Registrar gasto espontáneo	MK-009 Daily Input	DCA-002 Balance y Daily Input	SD-007 Registrar Movimiento		ENTM-06 SpontaneousSpending	ENTR-06 SpontaneousSpending	(Pendiente)	(Pendiente)	(Pendiente)	CD-INT-04	(Pendiente)	(Pendiente)		(Pendiente)
RF-07	El sistema calculará automáticamente los totales de ingresos, gastos y ahorros, basándose en las entradas del usuario.	Funcional	CU-07 Registrar movimiento (general)	MK-009 Daily Input	DCA-002 Balance y Daily Input	SD-007 Registrar Movimiento		ENTM-02 Account ENTM-05 Movement	ENTR-02 Account ENTR-05 Movement	(Pendiente)	(Pendiente)	(Pendiente)	CD-INT-04	(Pendiente)	(Pendiente)		(Pendiente)
RF-08	El sistema permitirá generar reportes financieros detallados que los usuarios podrán exportar a formatos PDF o HTML.	Funcional	CU-09 Generar reporte	MK-006 Download Report	DCA-002 Balance y Daily Input	SD-002 Generar Reporte		ENTM-02 Account ENTM-04 Concept ENTM-05 Movement ENTM-06 SpontaneousSpending	ENTR-02 Account ENTR-04 Concept ENTR-05 Movement ENTR-06 SpontaneousSpending	(Pendiente)	(Pendiente)	(Pendiente)	(Pendiente)	(Pendiente)	(Pendiente)		(Pendiente)
RF-09	El sistema permitirá a los usuarios definir una meta de ahorro mensual y anual.	Funcional	CU-04 Crear meta, CU-05 Editar meta	MK-008 Metas	DCA-003 Metas	SD-006 Crear Meta		ENTM-02 Account ENTM-03 Goals	ENTR-02 Account ENTR-03 Goals	FP-14 createObjetivo FP-31 createObjetivo FP-30 fetchNextNumeroObjetivo FP-33 fetchDetalle FP-34 updateEstadoMeta	COMP-02 GestionMetas	(Pendiente)	(Pendiente)	(Pendiente)	(Pendiente)		CD-SIS-05
RF-10	El sistema mostrará gráficamente el progreso de ahorro del usuario hacia la meta establecida.	Funcional	CU-06 Consultar progreso de meta	MK-008 Metas	DCA-003 Metas	SD-008 Consultar Progreso Meta		ENTM-02 Account ENTM-03 Goals	ENTR-02 Account ENTR-03 Goals	FP-29 fetchObjetivos FP-32 calculateFakeProgress	COMP-02 GestionMetas	(Pendiente)	(Pendiente)	(Pendiente)	(Pendiente)		CD-SIS-05
RNF-01	El sistema debe garantizar la seguridad mediante encriptación de contraseñas y protección de datos.	No Funcional	CU-01 Registrarse CU-02 Iniciar Sesión CU-03 Recuperar Contraseña	MK-001 Login MK-002 Sign Up MK-010 Forgot Password	DCA-001 Login y SignUp	SD-004 Iniciar Sesión SD-003 Registrarse SD-005 Recuperar Contraseña		ENTM-01 User ENTM-02 Account ENTM-03 Goal ENTM-04 Concept ENTM-05 Movement ENTM-06 SpontaneousSpending	ENTR-01 User ENTR-02 Account ENTR-03 Goal ENTR-04 Concept ENTR-05 Movement ENTR-06 SpontaneousSpending	(Pendiente)	(Pendiente)	(Pendiente)	(Pendiente)	(Pendiente)	(Pendiente)		(Pendiente)
RNF-02	El sistema debe ser capaz de realizar el inicio de sesión de manera eficiente, sin demoras perceptibles.	No Funcional	CU-02 Iniciar sesión	MK-001 Login	DCA-001 Login y SignUp	SD-004 Iniciar Sesión		ENTM-01 User	ENTR-01 User	(Pendiente)	(Pendiente)	(Pendiente)	(Pendiente)	(Pendiente)	(Pendiente)		(Pendiente)
RNF-03	El sistema debe garantizar que solo los usuarios debidamente autenticados y autorizados puedan recuperar o restablecer su contraseña. Para ello, el proceso de recuperación debe incluir mecanismos de validación de identidad robustos evitando que terceros no autorizados accedan a las credenciales del usuario.	No Funcional	CU-03 Recuperar contraseña	MK-010 Forgot Password	DCA-001 Login y SignUp	SD-005 Recuperar Contraseña		ENTM-01 User	ENTR-01 User	(Pendiente)	(Pendiente)	(Pendiente)	(Pendiente)	(Pendiente)	(Pendiente)		CD-SIS-06
RNF-04	El sistema debe ofrecer una alta disponibilidad y un tiempo de respuesta eficiente al generar balances financieros. La visualización de reportes, gráficos y resúmenes de gastos/ingresos debe realizarse sin demoras perceptibles para el usuario.	No Funcional	CU-06 Consultar progreso de meta CU-08 Consultar balance CU-09 Generar Reporte	MK-005 Balance MK-006 Download Report MK-008 Metas	DCA-002 Balance y Daily Input DCA-003 Metas	SD-001 Consultar Balance SD-002 Generar Reporte SD-008 Consultar Progreso Meta		ENTM-02 Account ENTM-03 Goal ENTM-04 Concept ENTM-05 Movement ENTM-06 SpontaneousSpending	ENTR-02 Account ENTR-03 Goal ENTR-04 Concept ENTR-05 Movement ENTR-06 SpontaneousSpending	(Pendiente)	(Pendiente)	(Pendiente)	(Pendiente)	(Pendiente)	(Pendiente)		(Pendiente)

Acceso a la Matriz de Trazabilidad

1. Para acceder a la Matriz de Trazabilidad simplemente debe acceder al repositorio de github, en la carpeta “Otros Documentos”
2. Link Proporcionado: <https://github.com/ZeroDyna/Planify>

Análisis

3. Análisis Planify

La presente sección tiene como objetivo analizar en profundidad el sistema Planify. El análisis permite identificar los principales actores del sistema, los requerimientos funcionales y no funcionales, y los requerimientos de usuario, así como los casos de uso que guiarán el desarrollo del software.

El propósito de esta sección es establecer una base sólida para el diseño y construcción del sistema, asegurando una comprensión clara de cómo debe comportarse la aplicación frente a distintos escenarios de uso.

a. Requerimientos

En esta sección se presentan los requerimientos del sistema clasificados en tablas separadas por tipo y color:

- **Requerimientos funcionales** (color rojo)
- **Requerimientos no funcionales** (color verde)
- **Requerimientos de usuario** (color azul)

Cada tabla detalla un requisito específico con su justificación, alcance, restricciones y consideraciones adicionales, garantizando una comprensión clara de las funcionalidades y características que el sistema debe ofrecer.

i. Requerimientos de Usuario

En esta sección se detallan los requerimientos de usuario del sistema. Cada tabla presenta un requisito específico, su justificación, alcance, restricciones y consideraciones adicionales, con el objetivo de garantizar una comprensión clara de las funcionalidades que la aplicación debe ofrecer para satisfacer las necesidades de los usuarios

Tabla 01. Registro y Gestión de cuenta (RU-01)

Descripción	El sistema debe permitir que los usuarios se registren con nombre completo, login y contraseña, y posteriormente puedan actualizar sus datos básicos desde la sección Account.
Justificación	El registro es la puerta de entrada para acceder a los servicios y personalizar la experiencia en la aplicación.
Alcance	Disponible para todos los usuarios nuevos que deseen acceder al sistema.

Tabla 01. Continuación desde la página anterior

Restricciones	El registro solo será posible si el usuario proporciona la información mínima requerida y cumple con los requisitos del sistema.
Consideraciones adicionales	Los datos deben almacenarse de forma segura y conforme a la normativa vigente de protección de datos.

Tabla 02. Eliminación de la Cuenta (RU-02)

Descripción	El sistema debe permitir a los usuarios solicitar la eliminación de su cuenta y de la información personal asociada.
Justificación	Garantiza el derecho de los usuarios a decidir sobre la permanencia de sus datos en la aplicación.
Alcance	Aplica a cualquier usuario registrado que lo solicite.
Restricciones	Una vez eliminada la cuenta, no será posible recuperar los datos ni registros financieros.
Consideraciones adicionales	Se debe solicitar confirmación expresa del usuario antes de proceder con la eliminación.

Tabla 03. Recuperación de acceso (RU-03)

Descripción	El sistema debe ofrecer un mecanismo para recuperar el acceso en caso de olvido de contraseña, mediante validación por correo electrónico.
Justificación	Asegura la continuidad del servicio y evita la pérdida de acceso por causas menores.
Alcance	Disponible para todos los usuarios registrados.
Restricciones	El proceso de recuperación requiere que el usuario tenga acceso al correo asociado a la cuenta.

Tabla 03. Continuación desde la página anterior

Consideraciones adicionales	La contraseña restablecida debe cumplir con políticas de seguridad.
------------------------------------	--

Tabla 04. Inicio de Sesión (RU-04)

Descripción	Los usuarios podrán iniciar sesión con su usuario y contraseña.
Justificación	Garantiza el acceso seguro y continuo de los usuarios a la plataforma.
Alcance	Aplica a todos los usuarios registrados.
Restricciones	El cambio de contraseña requiere validación por correo electrónico.
Consideraciones adicionales	La contraseña debe cumplir con políticas mínimas de seguridad.

Tabla 05. Visualización de Balance de Finanzas (RU-05)

Descripción	El sistema debe mostrar el balance actualizado de ingresos y gastos mensuales y anuales. Asimismo, da la opción de visualizar de acuerdo a un rango de fechas.
Justificación	Permite a los usuarios conocer el estado real de sus finanzas.
Alcance	Disponible para todos los usuarios con al menos un registro de ingresos o gastos.
Restricciones	Los cálculos dependen de la exactitud de los datos ingresados.
Consideraciones adicionales	La información se debe presentar de forma clara y entendible, con posibilidad de exportación en el futuro.

Tabla 06. Gestión de Metas de Ahorro (RU-06)

Descripción	El sistema debe permitir establecer metas anuales y mensuales de ahorro y mostrar gráficos de progreso.
Justificación	Motiva y orienta a los usuarios en su planificación financiera.

Tabla 06. Continuación desde la página anterior

Alcance	Disponible para todos los usuarios con cuenta activa.
Restricciones	El progreso se calcula en función de los ingresos y gastos ingresados en el sistema.
Consideraciones adicionales	Los gráficos deben ser interactivos y fáciles de interpretar.

Tabla 07. Ingreso Diario de Movimientos (RU-07)

Descripción	El sistema debe permitir registrar ingresos y gastos diarios, clasificados por concepto o categoría previamente configuradas por el usuario (ejemplo: transporte, comida, universidad) e incluir gastos espontáneos.
Justificación	Facilita el control financiero diario y la categorización de los movimientos.
Alcance	Disponible en todo momento para los usuarios registrados.
Restricciones	El usuario debe ingresar manualmente la información.
Consideraciones adicionales	El sistema debe permitir la edición y eliminación de registros.

Tabla 08. Configuración de Categorías (RU-08)

Descripción	El sistema debe permitir a los usuarios editar o agregar nuevos conceptos o categorías de ingresos y gastos.
Justificación	Ofrece flexibilidad y personalización en la gestión financiera.
Alcance	Disponible en la sección Configuración para todos los usuarios.
Restricciones	Las categorías no deben duplicarse con las ya existentes.
Consideraciones adicionales	Se debe asegurar consistencia de datos en los reportes al modificar categorías.

Tabla 09. Edición de Datos Personales (RU-09)

Descripción	El sistema debe permitir a los usuarios actualizar sus datos personales como el nombre, la dirección, trabajo y le ofrece al usuario la opción de cambiar contraseña.
Justificación	Asegura la vigencia de la información del usuario y la personalización de la experiencia.
Alcance	Aplica a todos los usuarios registrados.
Restricciones	Algunos campos podrán estar bloqueados por seguridad (ej. correo principal).
Consideraciones adicionales	Los cambios deben registrarse en el historial de auditoría del sistema.

ii. Requerimientos Funcionales

En esta sección se detallan los requerimientos funcionales del sistema Planify. Cada tabla presenta un requisito específico, su identificador, nombre, características, descripción, a qué requerimientos no funcionales está relacionado y la prioridad del requerimiento, cada requisito funcional asegura que el sistema cumpla con su propósito principal y permita al usuario interactuar de manera efectiva con la aplicación.

Tabla 10. Inicio de Sesión

Identificador	RF-01
Nombre	Inicio de Sesión
Características	Funcionalidad para que los usuarios inicien sesión con correo electrónico y contraseña.
Descripción	El sistema permitirá a los usuarios iniciar sesión utilizando su nombre de usuario y contraseña.
Requerimientos No Funcionales Relacionados	RNF-01, RNF-02
Prioridad	Alta

Tabla 11. Registro de Nuevo Usuario

Identificador	RF-02
----------------------	--------------

Nombre	Registro de Nuevo Usuario
---------------	----------------------------------

Tabla 11. Continuación desde la página anterior

Características	Funcionalidad para registrar nuevos usuarios mediante correo electrónico y contraseña.
Descripción	El sistema permitirá a los usuarios registrarse mediante su correo electrónico, nombre de usuario y contraseña.
Requerimientos No Funcionales Relacionados	RNF-01, RNF-02
Prioridad	Alta

Tabla 12. Recuperación de Contraseña

Identificador	RF-03
Nombre	Recuperación de Contraseña
Características	Funcionalidad para recuperar la contraseña mediante correo electrónico.
Descripción	El sistema permitirá a los usuarios recuperar su contraseña enviando un enlace al correo electrónico registrado.
Requerimientos No Funcionales Relacionados	RNF-01, RNF-02
Prioridad	Alta

Tabla 13. Balance por Rango de Fechas

Identificador	RF-04
Nombre	Balance por Rango de Fechas
Características	Funcionalidad para visualizar el balance según un rango de fechas.
Descripción	El sistema permitirá a los usuarios seleccionar un rango de fechas y visualizar el balance de ingresos y gastos dentro de ese período.
Requerimientos No Funcionales Relacionados	RNF-01, RNF-02
Prioridad	Alta

Tabla 14. Registro de ingresos y egresos por conceptos

Identificador	RF-05
Nombre	Registro de ingresos y egresos por conceptos
Características	Funcionalidad para registrar ingresos y gastos por categorías definidas por el usuario.
Descripción	Funcionalidad para registrar ingresos y gastos por categorías definidas por el usuario.
Requerimientos No Funcionales Relacionados	RNF-01, RNF-02
Prioridad	Alta

Tabla 15. Registro de ingresos y gastos espontáneos

Identificador	RF-06
Nombre	Registro de ingresos y gastos espontáneos
Características	Funcionalidad para registrar ingresos y gastos fuera de las categorías predefinidas.
Descripción	El sistema permitirá registrar gastos e ingresos no planificados fuera de las categorías preestablecidas.
Requerimientos No Funcionales Relacionados	RNF-01, RNF-02
Prioridad	Alta

Tabla 16. Cálculo automático de totales

Identificador	RF-07
Nombre	Cálculo automático de totales
Características	Funcionalidad para calcular automáticamente los totales de ingresos, gastos y ahorros.
Descripción	El sistema calculará automáticamente los totales de ingresos, gastos y ahorros, basándose en las entradas del usuario.
Requerimientos No Funcionales Relacionados	RNF-01, RNF-02
Prioridad	Alta

Tabla 17. Generación de reportes financieros

Identificador	RF-08
Nombre	Generación de reportes financieros
Características	Funcionalidad para generar reportes sobre los ingresos, gastos y ahorros del usuario.
Descripción	El sistema permitirá generar reportes financieros detallados que los usuarios podrán exportar a formatos PDF o HTML.
Requerimientos No Funcionales Relacionados	RNF-01, RNF-02
Prioridad	Alta

Tabla 18. Establecimiento de metas de ahorro

Identificador	RF-09
Nombre	Establecimiento de metas de ahorro
Características	Funcionalidad para que los usuarios establezcan metas de ahorro personalizadas.
Descripción	El sistema permitirá a los usuarios definir metas de ahorro, incluyendo el monto objetivo y la fecha límite.
Requerimientos No Funcionales Relacionados	RNF-01, RNF-02
Prioridad	Alta

Tabla 19. Visualización gráfica de progreso de ahorro

Identificador	RF-10
Nombre	Visualización gráfica de progreso de ahorro
Características	Funcionalidad para mostrar un gráfico que represente el progreso de ahorro hacia una meta establecida.
Descripción	El sistema mostrará gráficamente el progreso de ahorro del usuario hacia la meta establecida.
Requerimientos No Funcionales Relacionados	RNF-01, RNF-02
Prioridad	Alta

Tabla 20. Edición de Perfil

Identificador	RF-11
Nombre	Edición de Perfil
Características	Funcionalidad para editar la información del perfil (Nombre, lugar de trabajo, ubicación).
Descripción	El sistema actualiza los datos de la cuenta y los muestra en la pestaña de perfil.
Requerimientos No Funcionales Relacionados	RNF-03
Prioridad	Baja

iii. Requerimientos No Funcionales

En esta sección se detallan los requerimientos no funcionales de Planify. Cada tabla presenta un requisito específico, su identificador, nombre, características, descripción, a qué requerimientos no funcionales está relacionado y la prioridad del requerimiento. Los requerimientos no funcionales definen las características de calidad del sistema, estableciendo criterios como rendimiento, seguridad, disponibilidad, usabilidad y escalabilidad

Tabla 20. Seguridad de inicio de sesión

Identificador	RNF-01
Nombre	Seguridad
Descripción	El sistema debe garantizar la seguridad mediante encriptación de contraseñas y protección de datos.
Prioridad	Alta

Tabla 21. Rendimiento de inicio de sesión

Identificador	RNF-02
Nombre	Rendimiento
Descripción	El sistema debe ser capaz de realizar el inicio de sesión de manera eficiente, sin demoras perceptibles.
Prioridad	Alta

Tabla 22. Seguridad en recuperación de contraseña

Identificador	RNF-03
Nombre	Fiabilidad
Descripción	El sistema debe garantizar que solo los usuarios debidamente autenticados y autorizados puedan recuperar o restablecer su contraseña. Para ello, el proceso de recuperación debe incluir mecanismos de validación de identidad robustos evitando que terceros no autorizados accedan a las credenciales del usuario.
Prioridad	Alta

Tabla 23. Rendimiento en generación de balances

Identificador	RNF-04
Nombre	Disponibilidad
Descripción	El sistema debe ofrecer una alta disponibilidad y un tiempo de respuesta eficiente al generar balances financieros. La visualización de reportes, gráficos y resúmenes de gastos/ingresos debe realizarse sin demoras perceptibles para el usuario.
Prioridad	Alta

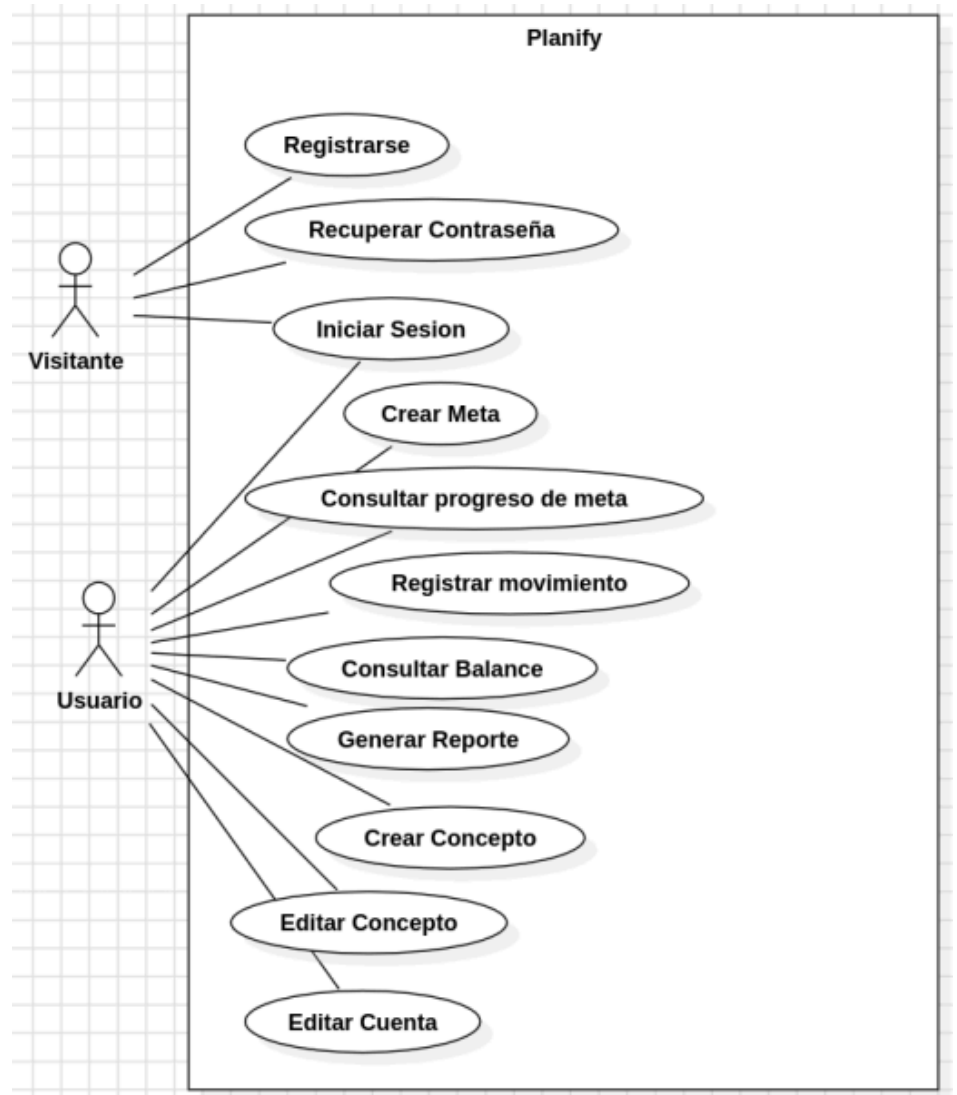
b. Modelo de Requisitos

i. Modelo de casos de uso

En esta sección se describen los principales casos de uso que conforman el comportamiento funcional del sistema. Los casos de uso permiten modelar y entender las interacciones entre los usuarios y el sistema, identificando los procesos esenciales que deben implementarse para cumplir con los objetivos del proyecto

Figura 1

Diagrama de Casos de Uso de Planify



Acceso al diagrama de casos de uso

1. Ingresar al repositorio de Planify en GitHub:
<https://github.com/ZeroDyna/Planify.git>
2. En la carpeta Diagramas, descargar el DiseñoPlanify.mdj.
3. Abrir la herramienta StarUML.
4. En el menú principal, seleccionar File > Open.y seleccionar el archivo descargado
5. En StarUml seleccionar la opción de Use Cases.

Especificaciones de caso de uso

CU-02 Iniciar sesión

Tabla 2.a. Especificaciones para el Caso de Uso “Iniciar sesión” (Descripción).

Fuente: propio.

Campo	Especificación
Caso de Uso	Iniciar sesión
Actores	Usuario
Requisito	RF-002-IS
Tipo	Permite a un usuario autenticarse en la plataforma ingresando credenciales válidas.
Precondición	El usuario debe estar registrado.
Poscondición	El usuario queda autenticado y puede acceder a las funcionalidades restringidas.

Tabla 2.b. Especificaciones para el Caso de Uso “Iniciar sesión” (Curso Normal).

Fuente: propio.

Curso Normal
1. El actor Usuario se encuentra en la Página de Inicio (Mk-001).
2. El Usuario selecciona la opción "Iniciar sesión" (Mk-001).
3. El sistema muestra el Formulario de Inicio de Sesión (Form-02) con campos obligatorios: Correo electrónico y contraseña.
4. El Usuario completa los campos y hace clic en el botón “Iniciar sesión”.
5. El sistema valida las credenciales: - Si son válidas: a. Crea la sesión de usuario. b. Registra evento de inicio de sesión (auditoría). c. Redirige a la Página Principal (Mk-001).

Tabla 2.c. Especificaciones para el Caso de Uso “Iniciar sesión” (Curso Alterno).

Fuente: propio.

Cursos Alternos
5.b. Credenciales inválidas: Mostrar mensaje de error “Correo o contraseña incorrectos”. Permitir reintentar.
5.c. Cuenta bloqueada/inactiva: Mostrar mensaje “Cuenta inactiva o bloqueada. Contacte al administrador”.

Tabla 2.c. Continuación desde la página anterior

5.d. Error de sistema: Mostrar mensaje “Servicio no disponible, intente más tarde”.

Formulario 2. Formulario de Inicio de Sesión

Campo	Descripción	Validaciones
Correo electrónico	Identificador único del usuario	No vacío, formato válido
Contraseña	Clave personal enmascarada	No vacía, mínimo 6 caracteres

Acceder a las especificaciones de casos de uso

El documento con las especificaciones detalladas de los casos de uso se encuentra disponible en el repositorio de Planify en GitHub. Puede accederse mediante el siguiente enlace en la carpeta Diccionarios y Especificaciones:

<https://github.com/ZeroDyna/Planify.git>

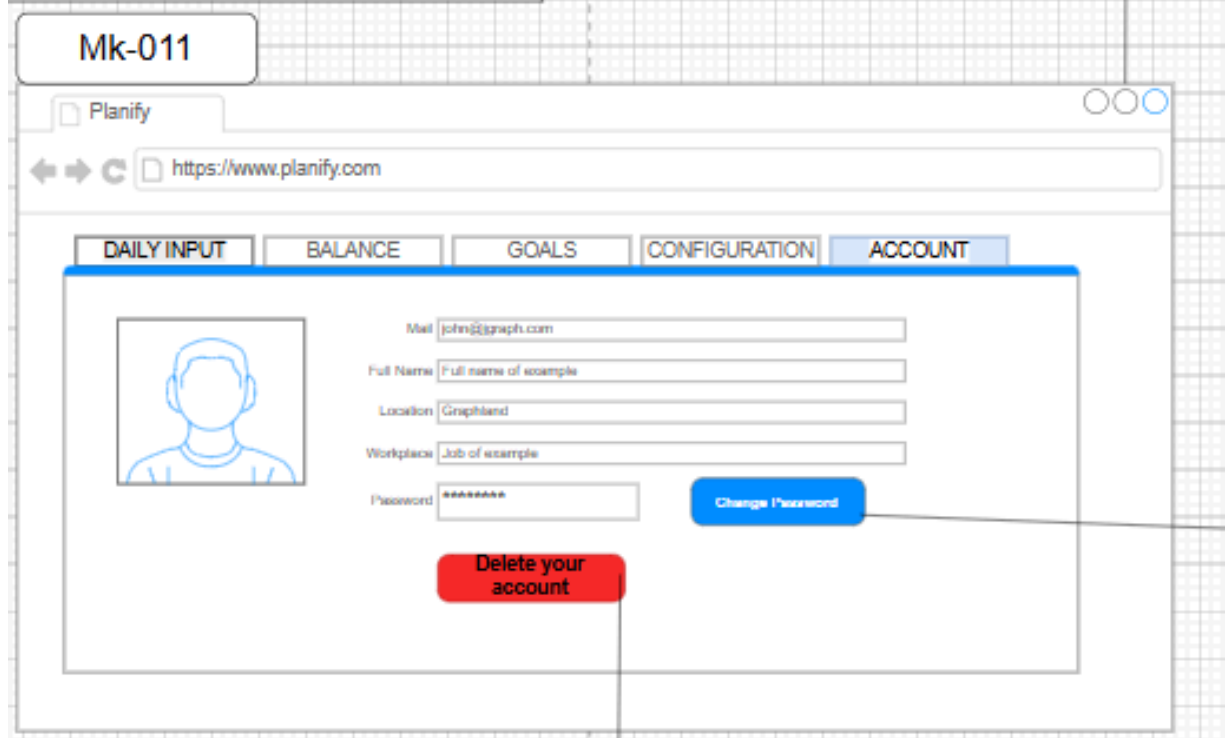
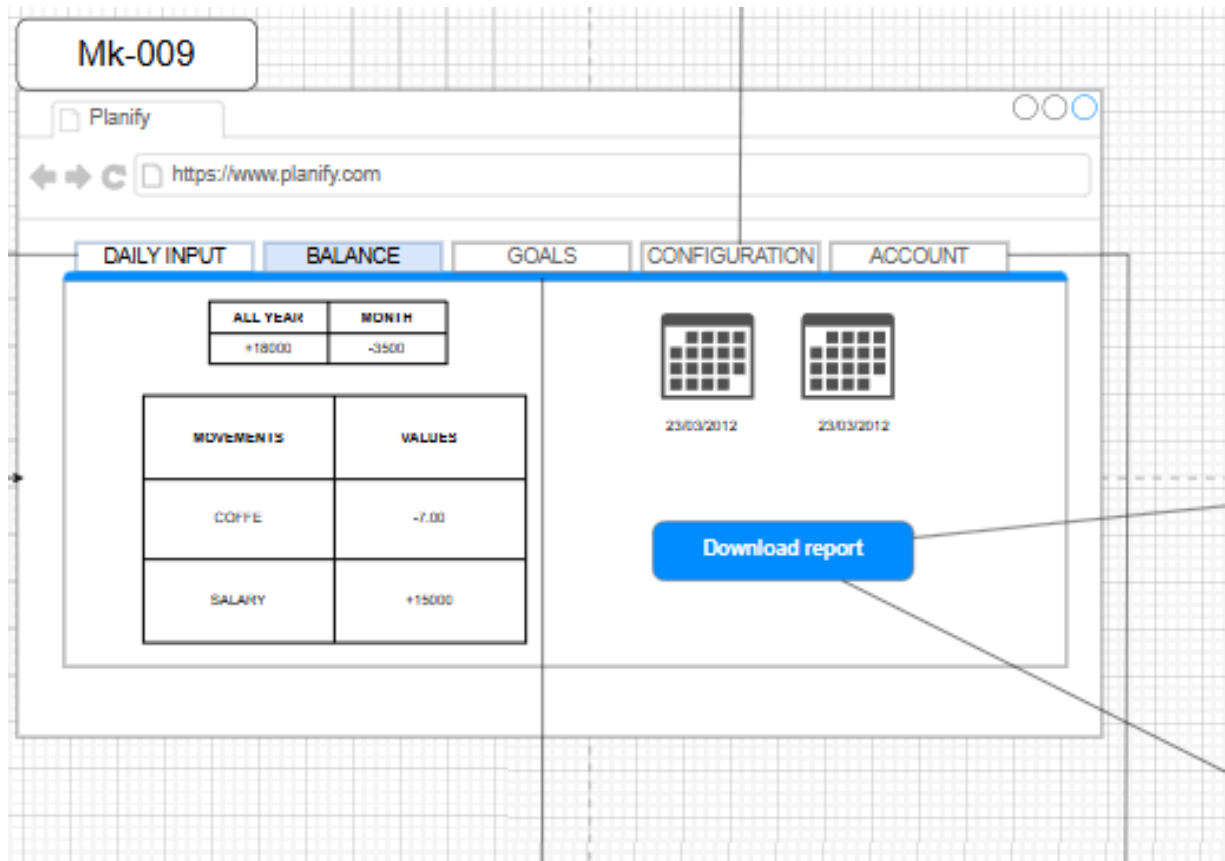
- Ingresar al enlace proporcionado.
- Seleccionar el archivo llamado **Casos de Uso y Especificaciones**.
- Descargar el archivo seleccionado.

ii. Modelo de presentación

En esta sección se muestran los modelos de presentación desarrollados para la plataforma Planify. Estos modelos representan visualmente las diferentes interfaces de usuario, permitiendo validar la experiencia del usuario y verificar el cumplimiento de los requerimientos funcionales del sistema.

Figura 2

Mockup de la interfaz de usuario Mk-009 para el módulo de balance financiero de la plataforma Planify y cómo se relaciona con el Mk-011



Acceso al Diagrama de Navegabilidad

- Acceder al link de Drawio proporcionado en [Link de Acceso](#) debajo de la Figura 2.
- Visualizar los mockups en Drawio.

Tabla 28. Especificación técnica y funcional de la interfaz Mk-009 del módulo Balance

ID	Nombre	Descripción Detallada	Restricciones / Reglas / Validaciones
MK-009	Balance	Interfaz que representa el estado financiero resumido de una cuenta de usuario. Muestra al usuario la situación actual de ingresos, egresos y resultados netos, así como permitir la descarga de reportes en un rango de fechas.	Se necesitan datos ingresados en el rango solicitado por el usuario.

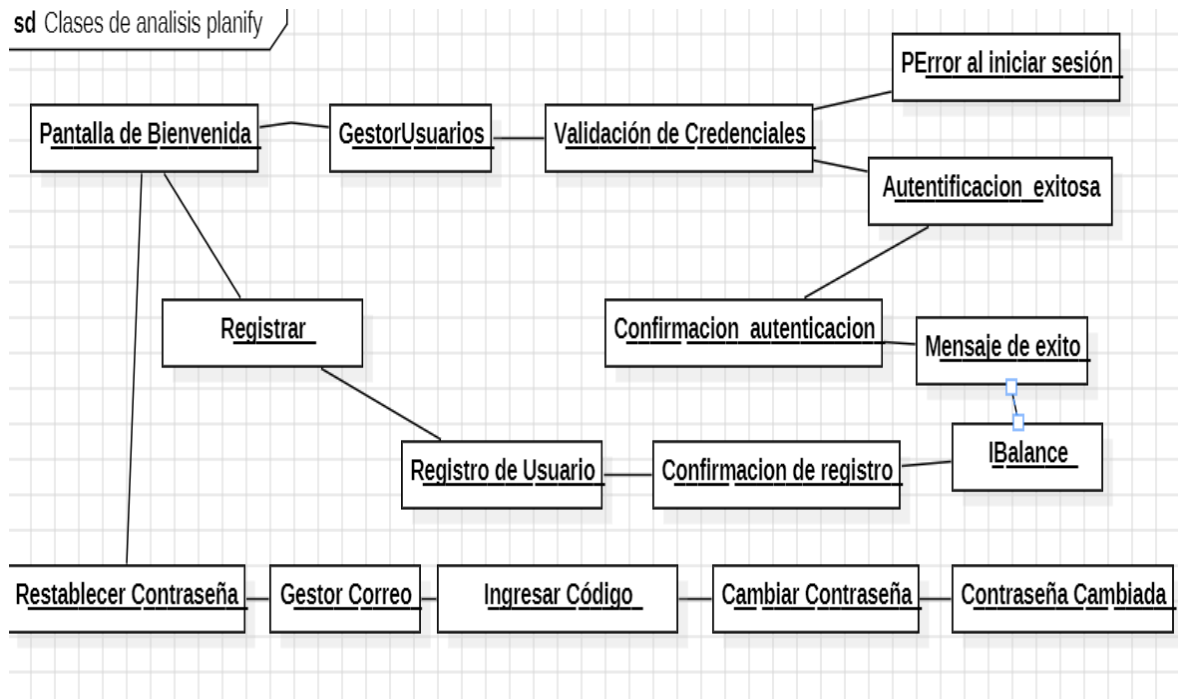
Acceso al Diccionario de Navegabilidad

- Acceder al link proporcionado [Diccionario de Navegabilidad](#)

ii. Información

En la Figura 3 se muestra el modelo de información representa la forma en que los distintos elementos del sistema interactúan entre sí para intercambiar datos y cumplir con los procesos definidos. En esta sección se presenta el diagrama de clases de análisis, el cual identifica los objetos del sistema y sus relaciones, así como el diagrama de comunicación que muestra las interacciones y comunicación entre estos objetos para realizar las funcionalidades definidas en los casos de uso. Estos diagramas se presentarán de forma completa y también organizados por módulos siguiendo la estructura de los Casos de Uso. Adicionalmente, se incluye un diccionario de términos para clarificar los conceptos utilizados.

Figura 3
Diagrama de Clases de Análisis



Acceso al diagrama de Clases de Análisis

1. Ingresar al repositorio de Planify en GitHub:
<https://github.com/ZeroDyna/Planify.git>
2. En la carpeta Diagramas, descargar el archivo Clases de analisis_1.mdj.
3. Abrir la herramienta StarUML.
4. En el menú principal, seleccionar File > Open.y seleccionar el archivo descargado.
5. En el explorador seleccionar Colaborationx/Interaction1/Clases de análisis planify x.
6. Diccionario del Diagrama de Clases de Análisis
Se encuentra como Metadata en la herramienta La tabla 29 define los objetos, interfaces y gestores presentes en el diagrama de comunicación, con su identificación y tipo correspondiente.

Tabla 29. Diccionario de Diagrama de Clases de Análisis

ID	Nombre	Tipo	Descripción	Comunicaciones
Mk-001	LoginUI	UI	Pantalla inicial donde el usuario ingresa credenciales para acceder al sistema Planify.	SignUpUI mediante botón SIGN UP. ChangeUI mediante Forgot Password?
O-01	:Login	Objeto	Representa la acción de iniciar sesión.	Objeto que representa la acción de iniciar sesión.
G-01	GestorAutenticacion	Gestor	Valida credenciales y gestiona la autenticación de usuarios.	Componente que valida credenciales y gestiona la autenticación de usuarios.
O-02	:SignUp	Objeto	Representa la acción de registro de nuevos usuarios.	Objeto que representa la acción de registro de nuevos usuarios.
Mk-011	SignUpUI	UI	Interfaz de registro de nuevos usuarios.	BalanceUI tras validarse la creación de cuenta.
G-02	GestorRegistro	Gestor	Procesa los datos de registro y crea nuevas cuentas.	Interfaz de registro de nuevos usuarios.(BalanceUI al validarse la creación de cuenta.
O-03	:Forgot	Objeto	Representa la acción de recuperar contraseña.	Objeto que representa la acción de recuperar contraseña.
Mk-002	ChangeUI	UI	Interfaz para el cambio o recuperación de contraseña.	Interfaz para el cambio o recuperación de contraseña.
G-03	GestorPassword	Gestor	Gestiona la lógica de recuperación y actualización de contraseñas.	Encargado de gestionar la lógica de recuperación y actualización de contraseñas.
Mk-006	BalanceUI	UI	Pantalla principal de balance y acceso al resto de funcionalidades.	DailyInputUI (pestaña DAILY INPUT). MetasUI (pestaña GOALS). ConfigUI (pestaña CONFIGURATION). CuentaUI (pestaña CUENTA). ReportUI (botón Download Report).

Tabla 29. Continuación desde la página anterior

Mk-014	DailyInputUI	UI	Interfaz para ingresar ingresos y gastos diarios.	Interfaz para ingresar ingresos y gastos diarios.
O-04	:DailyInput	Objeto	Representa la acción de introducir datos diarios.	- Objeto que representa la acción de introducir datos diarios.
G-04	GestorInput	Gestor	Procesa y almacena la información de los datos ingresados diariamente.	Procesa y almacena la información de los datos ingresados diariamente.
Mk-007	MetasUI	UI	Interfaz para la gestión y seguimiento de metas del usuario.	DailyInputUI (pestaña DAILY INPUT). BalanceUI (pestaña BALANCE). ConfigUI (pestaña CONFIGURATION). CuentaUI (pestaña CUENTA).
O-05	:Metas	Objeto	Representa la definición y administración de metas.	-Objeto que representa la definición y administración de metas.
G-05	GestorMetas	Gestor	Gestiona la creación, seguimiento y cumplimiento de metas.	Controlador que gestiona la creación, seguimiento y cumplimiento de metas.
Mk-015	ConfigUI	UI	Interfaz para configuración de los conceptos de ingresos y gastos.	DailyInputUI (pestaña DAILY INPUT). MetasUI (pestaña GOALS). BalanceUI (pestaña BALANCE). CuentaUI (pestaña CUENTA).
O-06	:Config	Objeto	Representa los parámetros de configuración de conceptos del sistema.	Objeto que representa los parámetros de conceptos del sistema.
G-06	GestorConfig	Gestor	Administra los cambios de configuración de conceptos del sistema.	Componente que administra los cambios de conceptos del sistema.

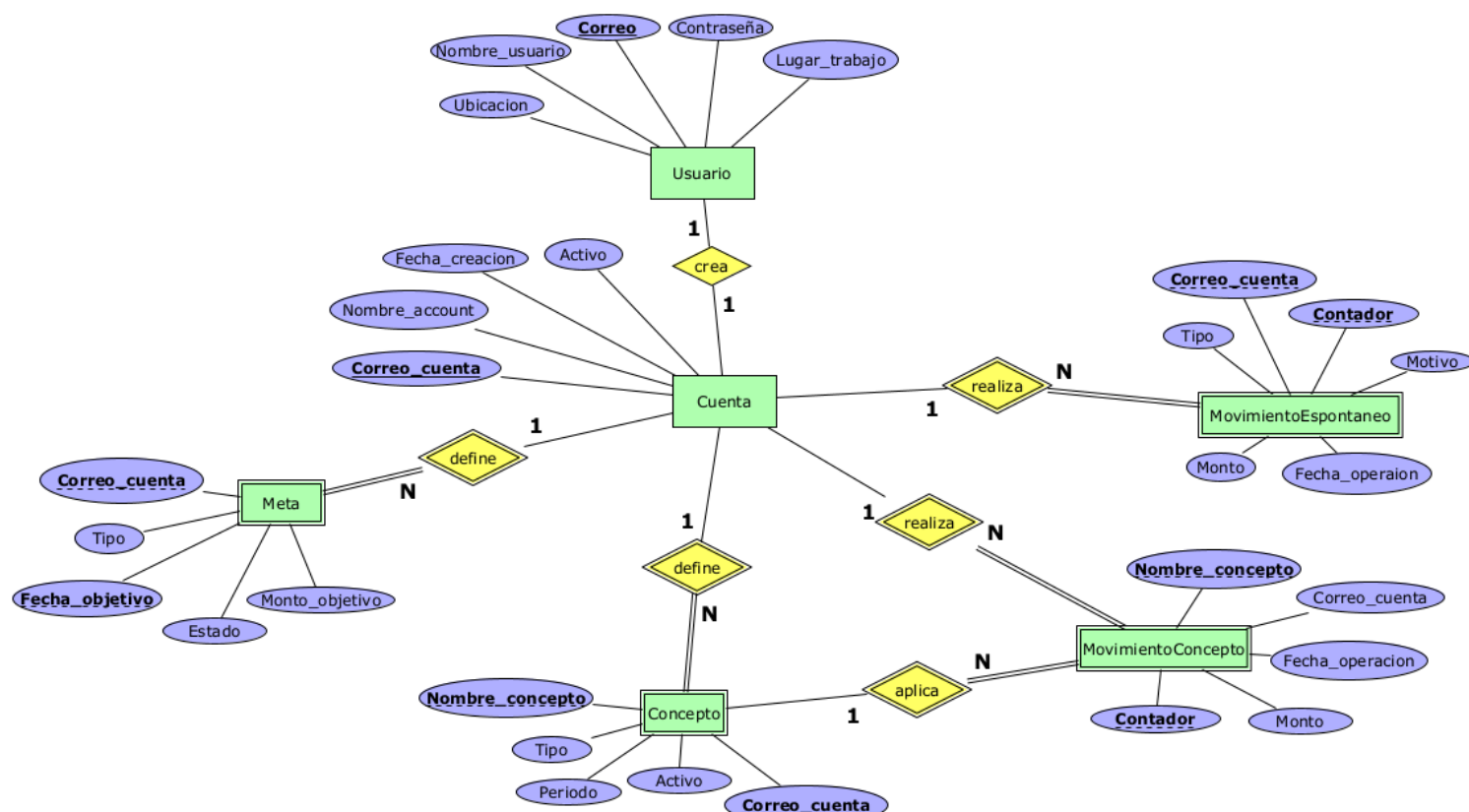
Tabla 29. Continuación desde la página anterior

Mk-008	CuentaUI	UI	Interfaz de gestión de la cuenta del usuario.	DailyInputUI (pestaña DAILY INPUT). MetasUI (pestaña GOALS). ConfigUI (pestaña CONFIGURATION). BalanceUI (pestaña BALANCE).
O-07	:Cuenta	Objeto	Representa los datos y operaciones de la cuenta del usuario.	Objeto que representa los datos y operaciones de la cuenta del usuario.
G-07	GestorCuenta	Gestor	Procesa cambios y operaciones relacionadas con la cuenta del usuario.	Procesa cambios y operaciones relacionadas con la cuenta.
Mk-010	ReportUI	UI	Interfaz que muestra un reporte generado a partir de las fechas especificadas.	Interfaz que muestra un reporte generado a partir de las fechas especificadas.
O-08	:Report	Objeto	Representa la acción de generar un reporte.	Objeto que representa la acción de generar un reporte.
G-08	GestorReporte	Gestor	Genera y presenta reportes con la información consolidada del sistema.	Encargado de generar y presentar reportes con la información del sistema.

c. Modelo Entidad Relación

El Diagrama Entidad Relación que corresponde a nuestro software “Planify”, a través del cual se puede observar cómo se relacionan las entidades en nuestra base de datos.

Figura 4
Modelo Entidad Relación



Acceso al modelo entidad relación

1. Ingresar al repositorio de Planify en GitHub:
<https://github.com/ZeroDyna/Planify.git>
2. Ingresar a la carpeta Diagramas
3. Descargar el archivo Modelo_entidad-relacion.xml.
4. Descargar y abrir la herramienta [TerraER](#).
5. En el menú principal, seleccionar File > Open. Y seleccionar el archivo descargado.

1. Diccionario de Diagrama entidad relación

Se describen formalmente las entidades, atributos y relaciones del Modelo Entidad–Relación del sistema PLANIFY, orientado a la gestión de cuentas y metas financieras personales.

Entidad: Usuario

Tipo de entidad: *Fuerte*

Justificación: Posee una clave propia (correo) que la identifica de forma única en el sistema y no depende de ninguna otra entidad para su existencia.

Descripción:

Representa a la persona registrada en el sistema. Cada usuario puede gestionar una o varias cuentas, y dispone de credenciales de acceso individuales.

Tabla 1.1. Atributos

Nombre del atributo	Descripción	Tipo de dato (referencial)	Obligatorio	Restricciones / Observaciones
Correo	Correo electrónico único del usuario.	Texto (320)	Sí	Identificador principal.
Contraseña	Contraseña cifrada del usuario.	Texto	Sí	Almacenada de forma segura (hash).
Nombre_usuario	Nombre visible o alias del usuario.	Texto (80)	Sí	—
Ubicación	Ciudad o país de residencia.	Texto (120)	No	—
Lugar_trabajo	Lugar o institución donde trabaja.	Texto (120)	No	—

Tabla 1.2. Relaciones

Nombre de la relación	Entidad relacionada	Cardinalidad	Descripción
Crea	Cuenta	1:1	Un usuario puede crear una cuenta.

Entidad: Cuenta

Tipo de entidad: *Fuerte*

Justificación: Tiene su propio identificador (correo_account) y no depende de otra entidad para su existencia lógica.

Descripción:

Representa el espacio financiero perteneciente a un usuario.

En una cuenta se definen los conceptos, objetivos y movimientos asociados.

Tabla 2.1. Atributos

Nombre del atributo	Descripción	Tipo de dato (referencial)	Obligatorio	Restricciones / Observaciones
Correo_cuenta	Correo o identificador único de la cuenta.	Texto (320)	Sí	Valor único.
Nombre_cuenta	Alias o nombre de la cuenta.	Texto (80)	Sí	—
Fecha_creacion	Fecha de creación de la cuenta.	Fecha/Hora	Sí	Generada automáticamente.
Activo	Estado operativo de la cuenta.	Booleano	Sí	TRUE = activa, FALSE = inactiva.

Tabla 2.2. Relaciones

Nombre de la relación	Entidad relacionada	Cardinalidad	Descripción
Pertenece a	Usuario	N:1	Cada cuenta pertenece a un único usuario.
Define	Objetivo	1:N	Una cuenta puede definir varios objetivos financieros.

Tabla 2.2. Continuación desde la página anterior

Define	Concepto	1:N	Una cuenta puede definir varios conceptos.
Registra	Movimiento_espontaneo	1:N	Una cuenta puede registrar múltiples movimientos espontáneos.
Registra	Movimiento_concepto	1:N	Una cuenta puede registrar múltiples movimientos asociados a conceptos.

Entidad: Objetivo

Tipo de entidad: *Fuerte*

Justificación: Aunque depende de una cuenta para su contexto funcional, posee identidad propia dentro del sistema.

Descripción:

Corresponde a un propósito financiero definido por el usuario dentro de una cuenta, como ahorro, inversión o pago de deuda.

Tabla 3.1. Atributos

Nombre del atributo	Descripción	Tipo de dato (referencial)	Obligatorio	Restricciones / Observaciones
Correo_cuenta	Cuenta a la que pertenece el objetivo.	Texto (FK)	Sí	Referencia a Cuenta.
Tipo	Tipo de objetivo (Ahorro, Deuda, Inversión, Emergencia).	Texto	Sí	Valor controlado.
Fecha_objetivo	Fecha límite estimada.	Fecha	No	—

Tabla 3.1. Continuación desde la página anterior

Estado	Estado actual del objetivo.	Texto	Sí	Pendiente, En progreso, Logrado, Cancelado
Monto_objetivo	Valor que se desea alcanzar.	Numérico	Sí	Mayor que cero.

Tabla 3.2. Relaciones

Nombre de la relación	Entidad relacionada	Cardinalidad	Descripción
Pertenece a	Cuenta	N:1	Cada objetivo pertenece a una cuenta.
Es definido por	Cuenta	N:1	La cuenta define una o varias metas u objetivos.

Entidad: Concepto

Tipo de entidad: *Fuerte*

Justificación: Tiene existencia independiente dentro del contexto de una cuenta y se utiliza como referencia en otras entidades (movimientos).

Descripción:

Categoría o clasificación creada por el usuario dentro de una cuenta para agrupar movimientos planificados de tipo ingreso o egreso.

Tabla 4.1. Atributos

Nombre del atributo	Descripción	Tipo de dato (referencial)	Obligatorio	Restricciones / Observaciones
Correo_cuenta	Cuenta a la que pertenece el concepto.	Texto (FK)	Sí	Referencia a Cuenta.

Tabla 4.1. Continuación desde la página anterior

Nombre_concepto	Nombre del concepto dentro de la cuenta.	Texto (60)	Sí	No se repite dentro de la misma cuenta.
Tipo	Naturaleza del concepto.	Texto	Sí	INGRESO o EGRESO.
Periodo	Periodicidad esperada.	Texto	No	Diario, Semanal, Mensual, etc.
Activo	Estado operativo del concepto.	Booleano	Sí	TRUE = activo, FALSE = inactivo.

Tabla 4.2. Relaciones

Nombre de la relación	Entidad relacionada	Cardinalidad	Descripción
Pertenece a	Cuenta	N:1	Cada concepto pertenece a una cuenta.
Aplica en	Movimiento_concepto	1:N	Un concepto puede aplicarse en varios movimientos.

Entidad: Movimiento espontáneo

Tipo de entidad: *Débil*

Justificación: Su existencia depende de la entidad Cuent.

No posee una clave propia global; se identifica mediante un **contador local** único por cuenta.

Por ello, su clave principal está compuesta por (Correo_account, Contador).

Descripción:

Registro de un movimiento financiero no asociado a un concepto. Representa ingresos o egresos puntuales o imprevistos.

Tabla 5.1. Atributos

Nombre del atributo	Descripción	Tipo de dato (referencial)	Obligatorio	Restricciones / Observaciones
Correo_cuenta	Cuenta donde ocurre el movimiento.	Texto (FK)	Sí	Referencia a Cuenta.
Contador	Número secuencial dentro de la cuenta.	Entero	Sí	Clave parcial; no se repite por cuenta.
Tipo	Tipo de movimiento.	Texto	Sí	INGRESO o EGRESO.
Motivo	Descripción del motivo del movimiento.	Texto (200)	No	—
Monto	Valor del movimiento.	Numérico	Sí	Mayor que cero.
Fecha_operacion	Fecha y hora del registro.	Fecha/Hora	Sí	—

Tabla 5.2. Relaciones

Nombre de la relación	Entidad relacionada	Cardinalidad	Descripción
Pertenece a	Cuenta	N:1	Cada movimiento espontáneo depende de una cuenta.
Identificado por	Cuenta	—	Usa la clave de cuenta más el contador local como clave compuesta.

Entidad: Movimiento concepto

Tipo de entidad: *Débil*

Justificación: Depende de Cuenta y de Concepto.

No tiene identidad propia; su clave está compuesta por (Correo_account, Contador) y su existencia se justifica sólo dentro de la cuenta a la que pertenece.

Descripción:

Registro de un movimiento financiero asociado a un concepto.

Permite realizar análisis de ingresos y egresos planificados.

Tabla 6.1. Atributos

Nombre del atributo	Descripción	Tipo de dato (referencial)	Obligatorio	Restricciones / Observaciones
Correo_account	Cuenta donde ocurre el movimiento.	Texto (FK)	Sí	Referencia a Cuenta.
Contador	Número secuencial dentro de la cuenta.	Entero	Sí	Clave parcial.
Nombre_concepto	Concepto asociado al movimiento.	Texto (FK)	Sí	Debe existir en la misma cuenta.
Monto	Valor del movimiento.	Numérico	Sí	Mayor que cero.
Fecha_operacion	Fecha y hora del registro.	Fecha/Hora	Sí	—

Tabla 6.2. Relaciones

Nombre de la relación	Entidad relacionada	Cardinalidad	Descripción
Pertenece a	Cuenta	N:1	Cada movimiento con concepto pertenece a una cuenta.

Tabla 6.2. Continuación desde la página anterior

Aplica a	Concepto	N:1	Cada movimiento se asocia a un concepto existente.
Identificado por	Cuenta	—	Usa la clave de cuenta más el contador local como clave compuesta.

Diseño

4. Diseño

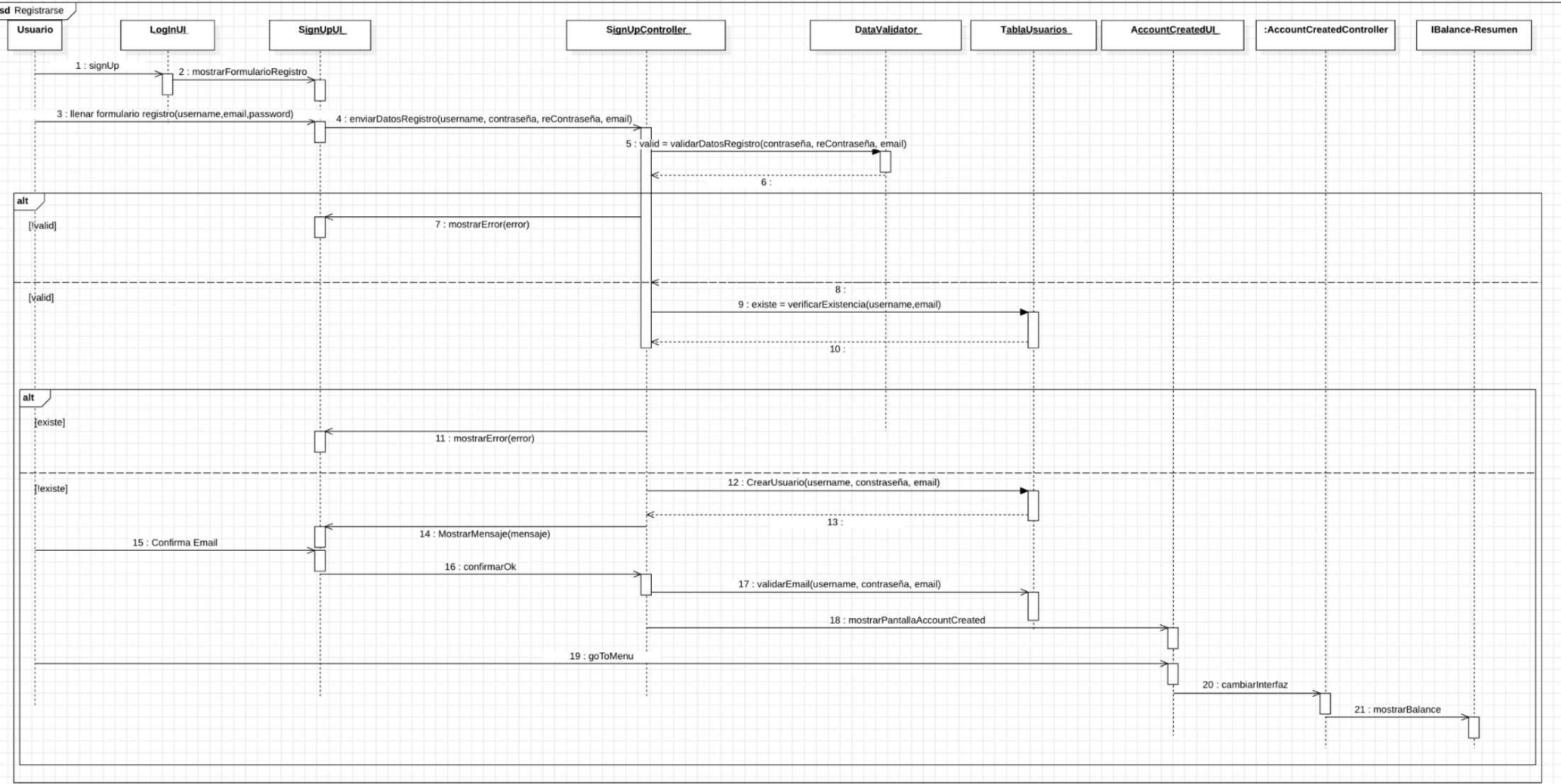
En esta sección se define la estructura técnica del sistema Planify, partiendo de los modelos desarrollados en el análisis. Se presentan los diagramas de secuencia que muestran cómo interactúan los componentes del sistema en los distintos casos de uso, así como el diagrama de clases de diseño, que detalla las entidades, atributos y relaciones del sistema. El objetivo es establecer una base sólida para la implementación, asegurando un funcionamiento eficiente y coherente con los requisitos definidos.

a. Diagramas de Secuencia

A continuación, se presentarán los diagramas de secuencia para cada Caso de Uso de nuestro software “Planify”. Cada diagrama representa cómo interactúan los objetos de nuestro sistema a lo largo del tiempo.

Figura 4

Diagrama de secuencia del caso de uso Registrarse (véase en la siguiente página)



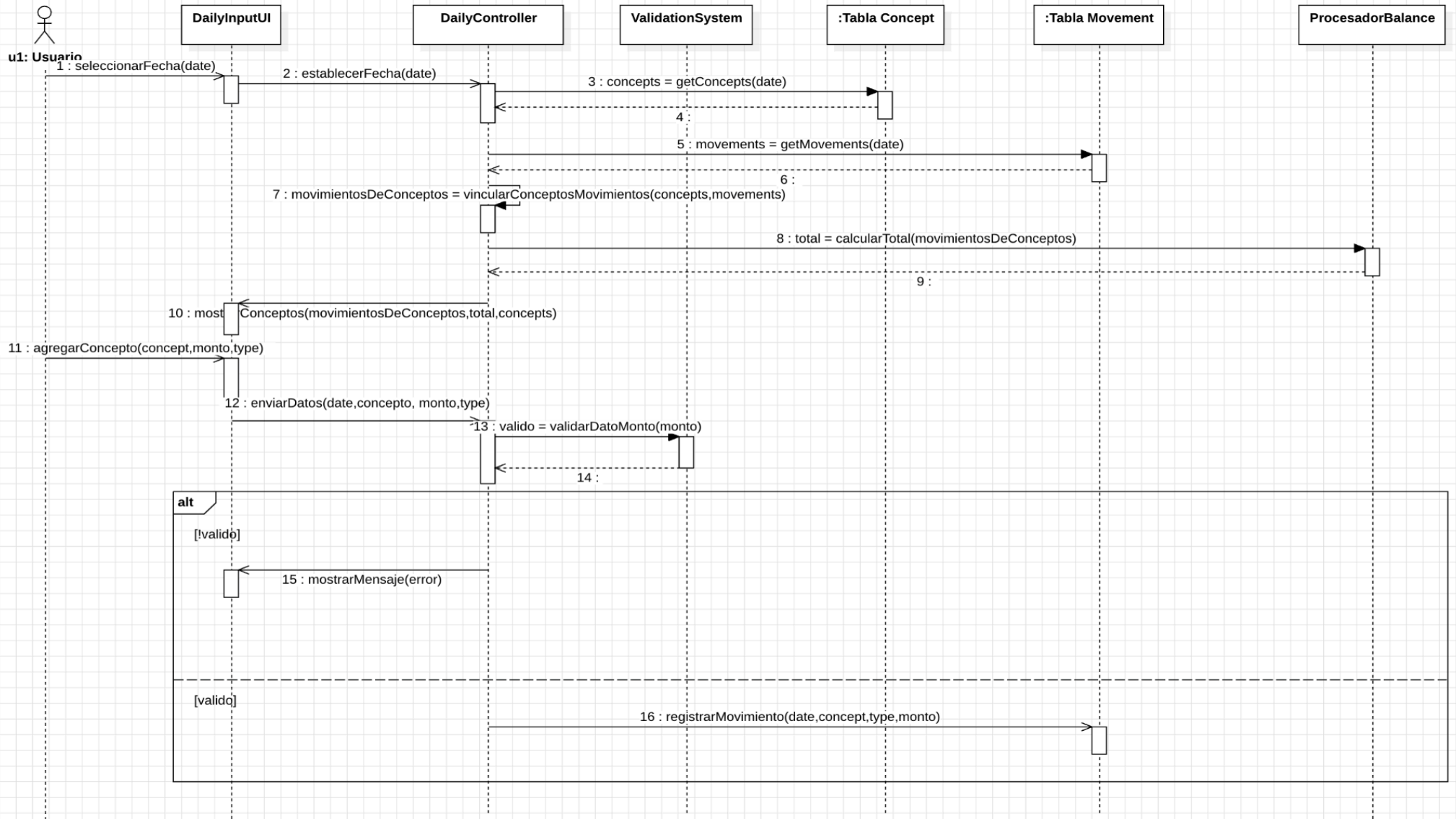
Acceso a los diagramas de secuencia

1. Ingresar al repositorio de Planify en GitHub:
<https://github.com/ZeroDyna/Planify.git>
2. En la carpeta Diagramas, descargar el DiseñoPlanify.mdj.
3. Abrir la herramienta StarUML.
4. En el menú principal, seleccionar File > Open. Y seleccionar el archivo descargado
5. En StarUml seleccionar la opción de Design Model/Collaboration/interaction.
6. Ahí se selecciona diagrama de secuencia deseado

Figura 5

Diagrama de secuencia del caso de uso Registrar Movimiento (véase en la siguiente página)

sd Registrar Movimiento

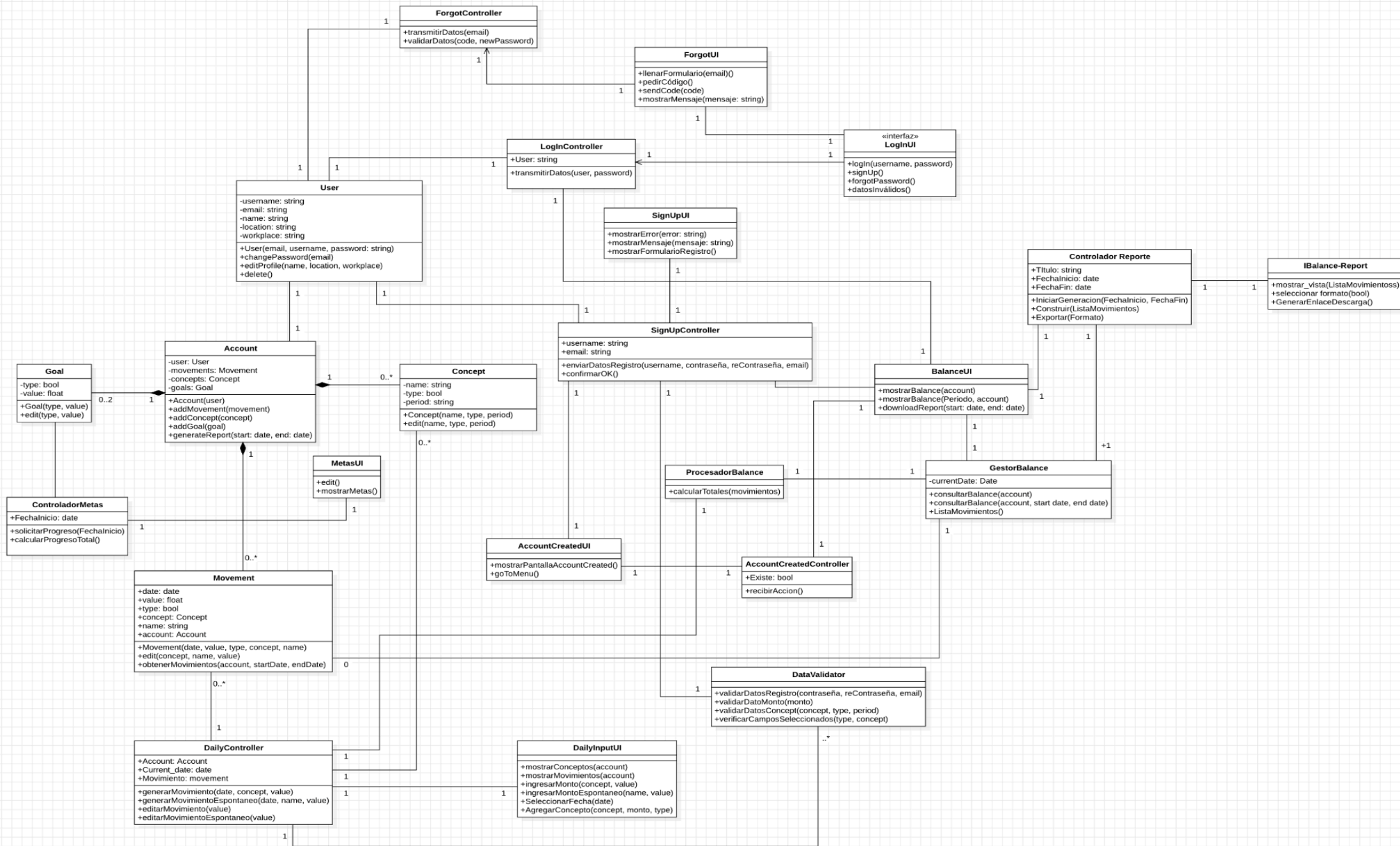


b. Diagrama de Clases de Diseño

En estos diagramas podemos observar las clases de nuestro sistema y las relaciones entre ellas. Gracias a estos diagramas podremos comprender mejor la visión general de los esquemas de nuestra aplicación, así como, entender visualmente las necesidades específicas de nuestro sistema. Los diagramas se encuentran separados por módulos de acuerdo a los Casos de Uso.

Figura 6

Diagrama de Clases de Diseño (ver en la siguiente página)



Archivo: Diagrama de Clases de Análisis (formato StarUML)

Acceso al Diagrama de Clases de Diseño

1. Ingresar al repositorio en GitHub usando el link proporcionado en <https://github.com/ZeroDyna/Planify.git> en la carpeta Diagramas, y descargar el archivo DiseñoPlanify.mdj.
2. Abrir el archivo descargado en la herramienta StarUML.
3. En el menú principal, seleccionar File > Open. Y seleccionar el archivo descargado.
4. En StarUml seleccionar la opción de Design Model/Diagrama de Clase.
5. Visualizar el Diagrama de Clases de Diseño en StarUML.

i. Diccionario del Diagrama de Clases de Diseño

Tabla 1. Diccionario de Clases principales

Nombre	User
Descripción	Esta clase representa al usuario registrado en la aplicación Planify. Contiene la información personal y de autenticación necesaria para acceder al sistema, así como los datos que permiten personalizar su experiencia dentro de la aplicación.

Atributo	Tipo de Dato	Visibilidad	Descripción
username	string	Privado	Nombre de usuario único utilizado para la autenticación dentro de Planify.
email	string	Privado	Correo electrónico del usuario. Sirve como identificador principal para inicio de sesión y recuperación de contraseña.
name	string	Privado	Nombre completo del usuario, visible en la sección de perfil.

Tabla 1.1. Continuación desde la página anterior

location	string	Privado	Ciudad y país del usuario. Permite personalizar la información de la cuenta.
workplace	string	Privado	Lugar de trabajo o institución asociada al usuario. Se muestra como parte de la información del perfil.

Método	Parámetro (tipo de dato)	Visibilidad	Tipo de Retorno	Descripción
User	email (string), username (string), password (string)	Público	void	Crea un nuevo usuario en el sistema con los datos proporcionados. Inicializa la información básica de perfil.
changePassword	email (string)	Público	bool	Permite al usuario cambiar su contraseña tras verificar su correo electrónico. Implementa la lógica de recuperación o actualización de credenciales.
editProfile	name,	Público	bool	Actualiza la

	location, workplace (string)			información del perfil del usuario con los nuevos datos ingresados.
--	------------------------------------	--	--	--

Acceso al diccionario del Diagrama de Clases de Diseño

1. Ingresar al [Repositorio](#).
2. En la carpeta Diccionarios y Especificaciones, visualizar el diccionario del Diagrama de Clases de Diseño.

c. Modelo Relacional

Este diagrama traduce nuestro Modelo Entidad-Relación (MER) en un esquema de base de datos formal. Define la estructura de las tablas, sus columnas, y las llaves primarias y foráneas que implementan las relaciones lógicas, sirviendo como el plano directo para la creación del modelo físico.

i. Algoritmo de mapeo de 7 pasos

1. Mapear entidades regulares (fuertes)

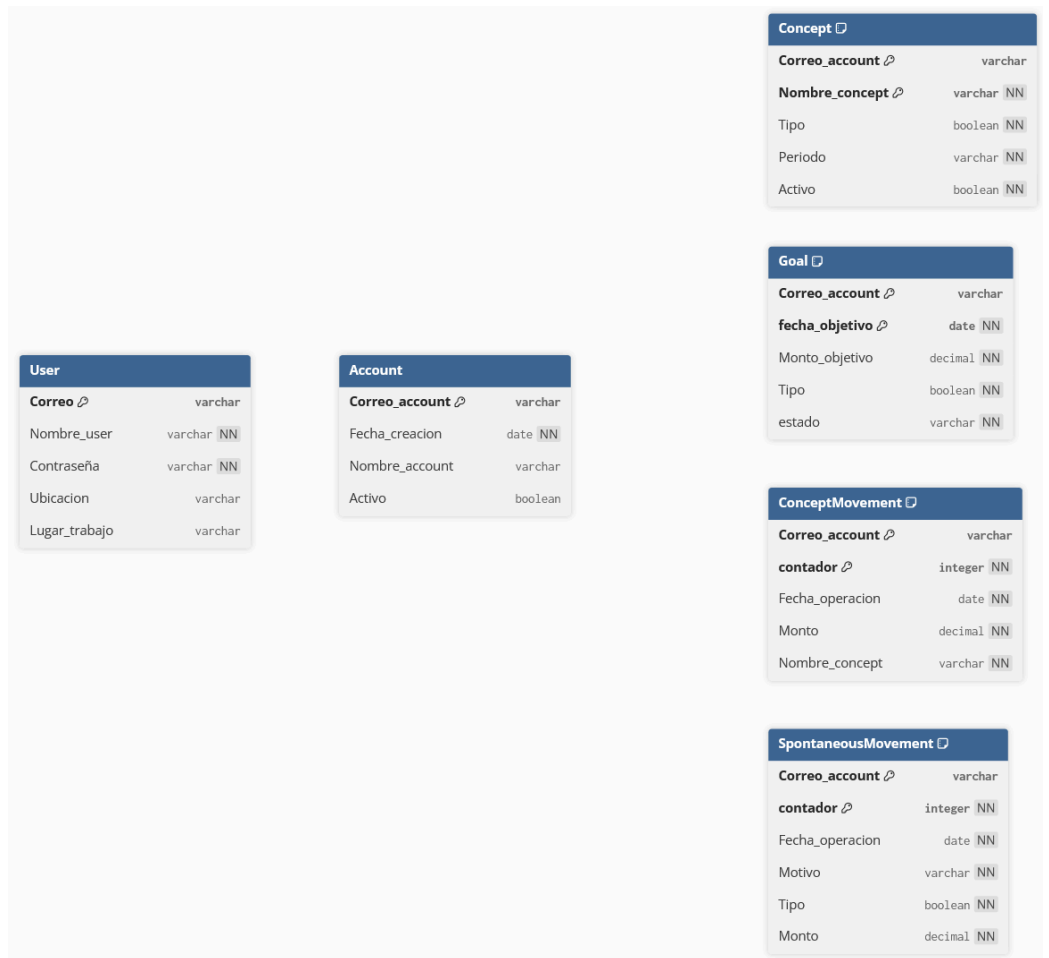
User, Account

User	Account
Correo 🔗 varchar	Correo_account 🔗 varchar
Nombre_user varchar NN	Fecha_creacion date NN
Contraseña varchar NN	Nombre_account varchar
Ubicacion varchar	Activo boolean
Lugar_trabajo varchar	

2. Mapear entidades débiles

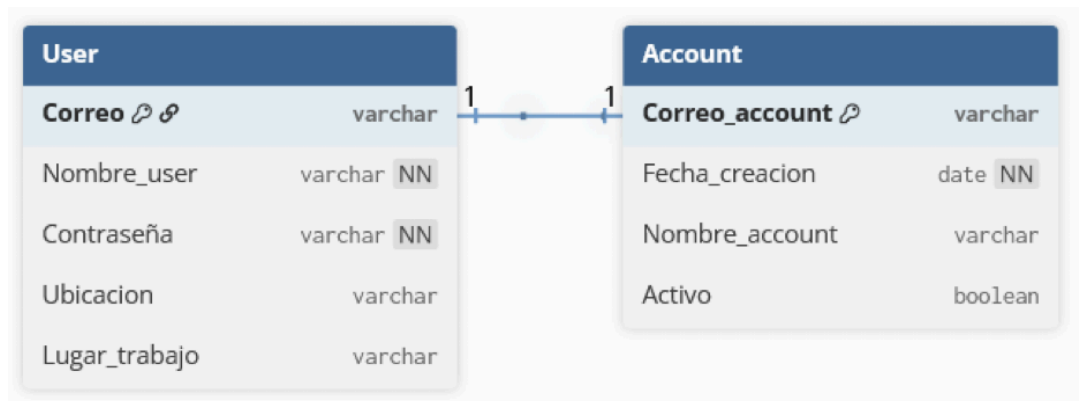
Concept, Goal, ConceptMovement, SpontaneusMovement3. Mapear relaciones uno a uno (1:1)

User – Account



3. Mapear relaciones uno a uno (1:1)

User – Account



4. Mapear relaciones uno a muchos (1:N)

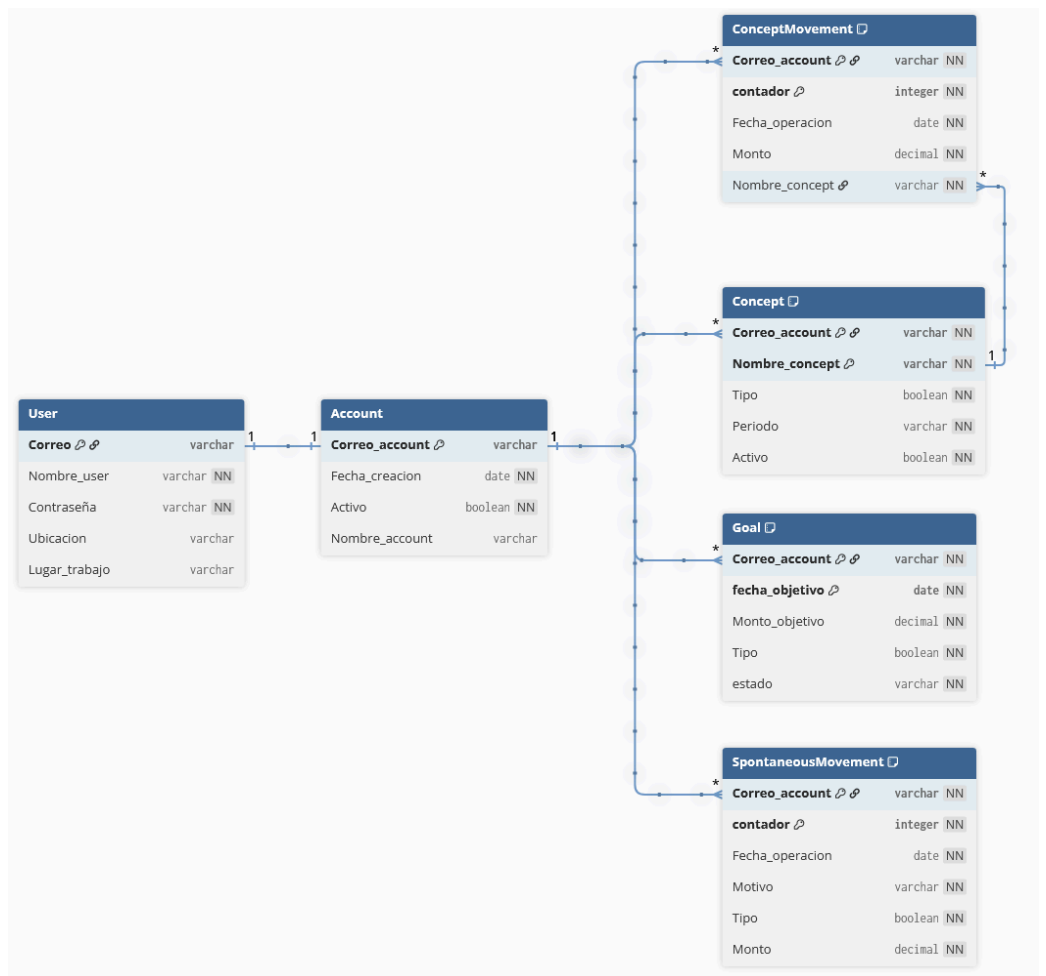
Account(1) – Goal(N)

Account(1) – Concept(N)

Account(1) – ConceptMovement(N)

Account(1) – SpontaneousMovement(N)

Concept(1) – ConceptMovement (N)



5. Mapear relaciones muchos a muchos (N:M)

No hay relaciones muchos a muchos.

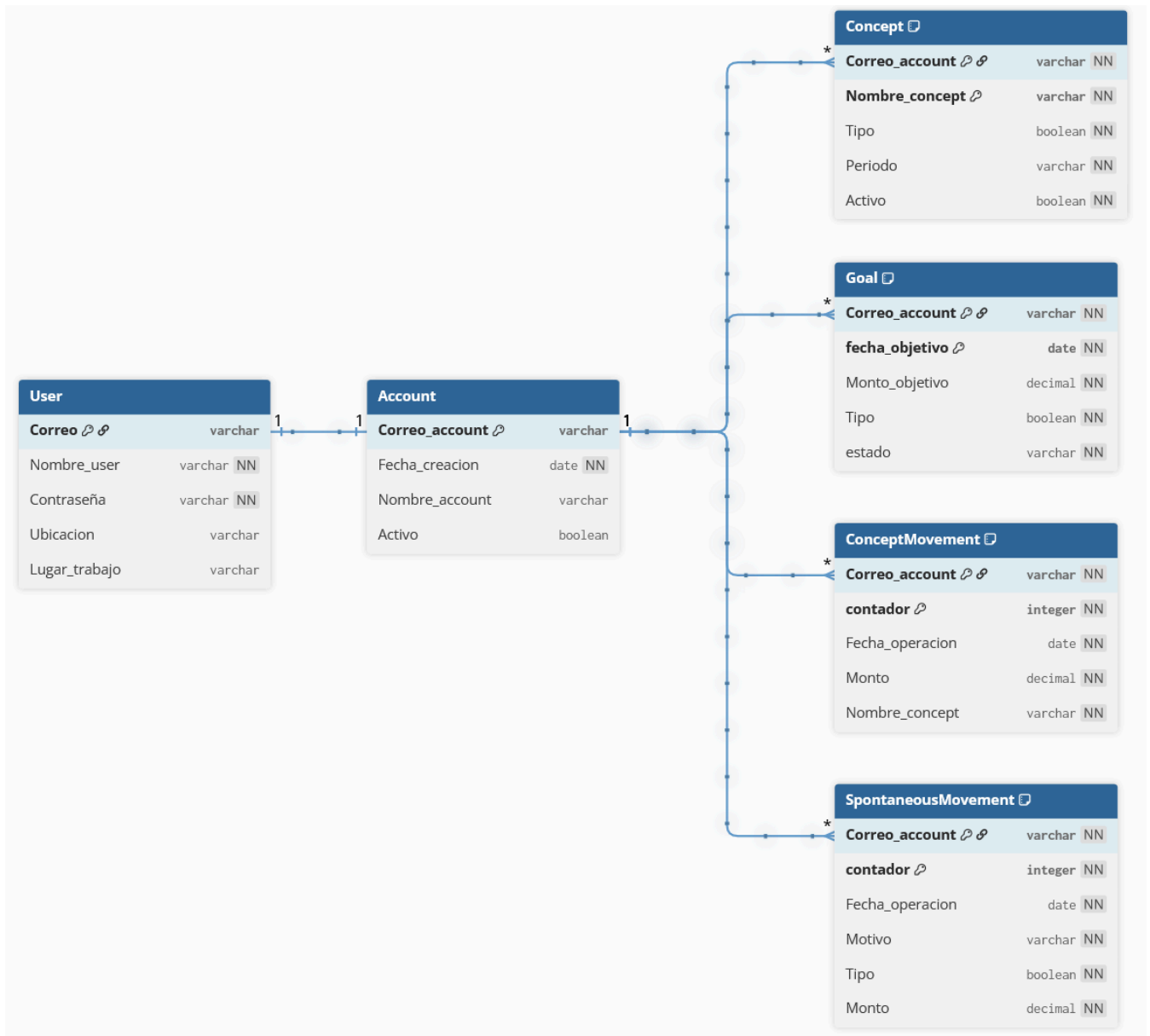
6. Mapear atributos multivaluados

No hay atributos multivalorados.

7. Mapear relaciones n-arias

No hay relaciones n-arias.

Figura 7
Modelo Relacional de Planify



Acceso al Modelo Relacional

1. Ingresar al repositorio de github de Planify (<https://github.com/ZeroDyna/Planify.git>)
2. En la carpeta Diagramas, descargar y abrir el archivo Diagrama_relacional.pdf

ii. Diccionario Modelo Relacional

El diccionario del Modelo Relacional para Planify detalla y define formalmente cada componente del diagrama relacional, explica el propósito de cada

tabla (derivada de una entidad), cada columna (atributo), y especifica las claves primarias (PK) y foráneas (FK) que implementan las relaciones en la base de datos.

Tabla 1. Entidad User

Tabla 1.1 Resumen de la Entidad

Elemento	Detalle
Nombre de la Entidad	User
Propósito/Descripción	Almacena la información de autenticación y personal de los usuarios del sistema.
Tipo de Entidad	Fuerte
Clave Primaria (PK)	Correo
Claves Foráneas (FK)	Ninguna.

Tabla 1.2 Detalle de Columnas (Atributos)

Columna	Tipo de Dato Lógico	Descripción	Restricciones / Reglas de Negocio	PK/FK	Acepta Nulos
Correo	Cadena de Caracteres	Correo electrónico único del usuario.	PK, Debe ser único y tener formato de email.	PK	No

Tabla 1.2. Continuación desde la página anterior

Nombre_ user	Cadena de Caracteres	Nombre visible del usuario.	NN.	-	No
Contraseña	Cadena de Caracteres	Hash de la contraseña del usuario.	NN. Debe ser almacenada como hash seguro.	-	No
Ubicación	Cadena de Caracteres	Ubicación de residencia del usuario.	Opcional.	-	Sí
Lugar_ trabajo	Cadena de Caracteres	Lugar o empresa de trabajo del usuario.	Opcional.	-	Sí

Acceso al Diccionario Diagrama del Modelo Relacional

1. Para acceder al Diccionario del Diagrama del Modelo Relacional completo en un Google Docs, se debe ingresar a través del siguiente enlace proporcionado
2. Link Proporcionado:
<https://docs.google.com/document/d/1UhvqO1ZTegSnIt5oUsItP6ECbOWkpWpV-cbkQfN9ZF4/edit?tab=t.0#heading=h.22b0ugqopq13>

d. Arquitecturas

La sección de arquitecturas describe la organización estructural del sistema, especificando sus capas, componentes y mecanismos de interacción. Su objetivo es definir el modelo técnico que soporta la distribución de responsabilidades, la comunicación interna y las propiedades de calidad del sistema, como rendimiento, escalabilidad y mantenibilidad.

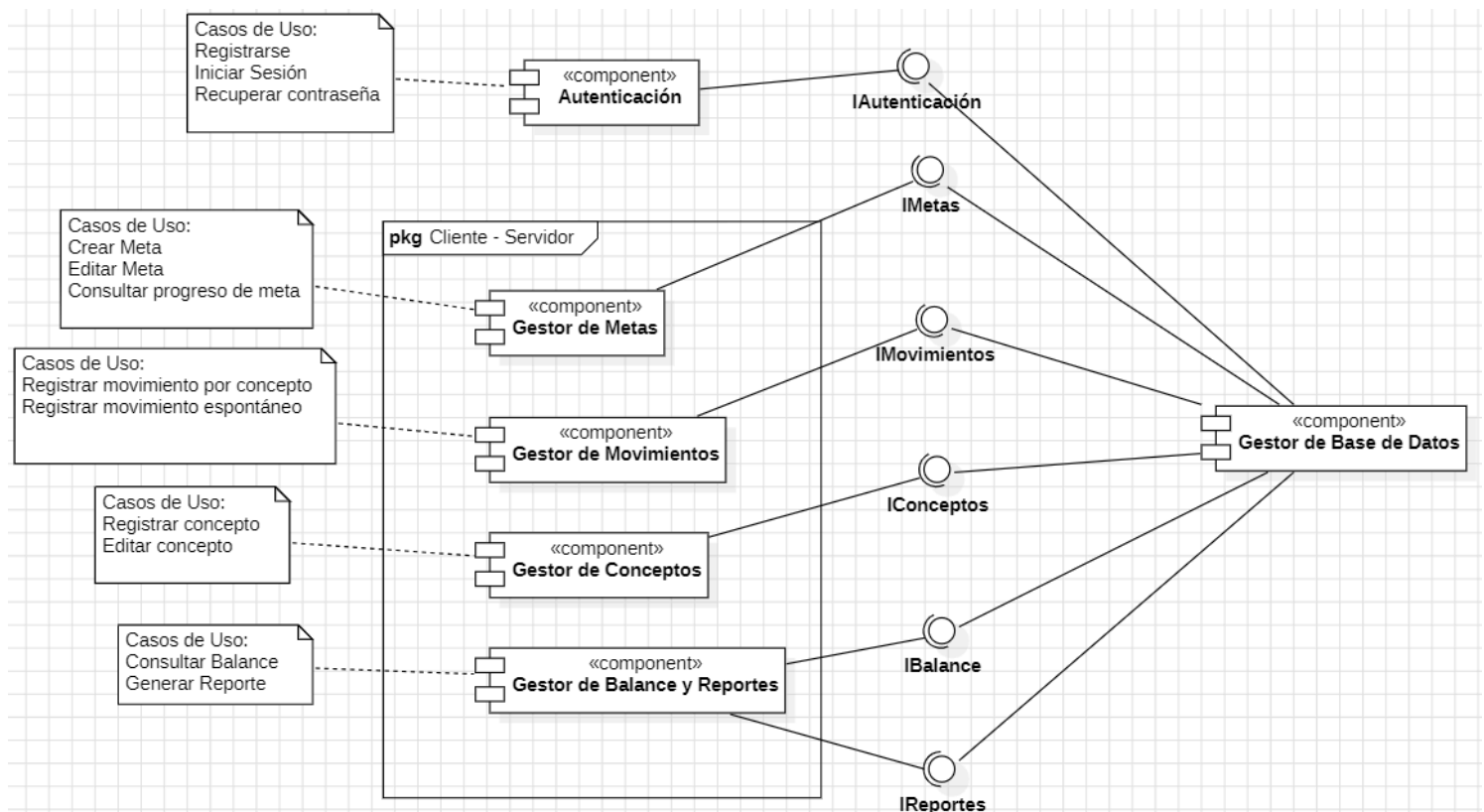
i. Diagrama de Componentes

En este diagrama se representan los principales módulos lógicos del sistema y la forma en que interactúan entre sí mediante interfaces claramente definidas. A través de la visualización de componentes, es posible comprender la estructura arquitectónica de la aplicación, identificar las responsabilidades de cada módulo y analizar cómo se comunican para cumplir con las funcionalidades descritas en los Casos de Uso.

El modelo se encuentra organizado en secciones que agrupan los componentes según las áreas funcionales del sistema, permitiendo una interpretación más clara de la arquitectura general y del rol que desempeña cada parte dentro de la plataforma.

Figura 8

Diagrama de Componentes de Planify



1. Acceso al Diagrama de Componentes

- Ingresar al repositorio de Planify en GitHub:
<https://github.com/ZeroDyna/Planify.git>
- En la carpeta Diagramas, descargar el DiseñoPlanify.mdj.
- Abrir la herramienta StarUML.
- En el menú principal, seleccionar File > Open. y seleccionar el archivo descargado
- En StarUml seleccionar la opción de Design Model.

f. Ahí se selecciona diagrama de componentes.

2. Diccionario del Diagrama de Componentes

En el diagrama de componentes del sistema, los módulos funcionales de Planify (Gestor de Metas, Gestor de Movimientos, Gestor de Conceptos, Gestor de Balance y Reportes, etc.) están agrupados dentro de un *package* llamado «**cliente-servidor**». Este paquete representa que cada componente lógico tiene una parte en el cliente (frontend React) y otra en el servidor (backend Node.js), compartiendo la misma responsabilidad funcional pero distribuida en capas. Por este motivo, en el diagrama de despliegue los mismos nombres de componentes aparecen repetidos en los diferentes nodos (Navegador Web y Node.js): se trata del **mismo componente lógico**, desplegado parcialmente en el cliente (capa de presentación) y parcialmente en el servidor (capa de lógica de negocio y acceso a datos). Esto aclara que la funcionalidad está dividida físicamente, pero pertenece al mismo módulo definido en el diagrama de componentes.

Tabla 1. Diccionario de Componentes – Gestor de Metas

Campo	Descripción
Nombre	Gestor de Metas
ID	COMP-01
Descripción	Componente responsable de administrar las metas financieras del usuario en el sistema Planify. Permite crear, editar y consultar metas, así como calcular el progreso y el cumplimiento de cada una a partir de los movimientos registrados.
Responsabilidades principales	• Registrar y actualizar metas de ahorro y gasto. • Calcular el porcentaje de avance de cada meta con base en los movimientos y configuraciones del usuario o familia. • Validar fechas límite, montos objetivos y estado de las metas (activa, cumplida, vencida). • Exponer datos de las metas a la interfaz web y a los reportes de balance. • Coordinar el acceso a la capa de persistencia mediante el Gestor de Base de Datos a través de la interfaz IMetas.

Nodos de despliegue	• PC Cliente (DESP-01) y Móvil Cliente (DESP-02) dentro del Navegador Web (DESP-06) como lógica de presentación. • Servidor Web (DESP-03) dentro del entorno de aplicación (DESP-07) como lógica de negocio y API REST.
Conexiones / Interfaces	• Consume servicios del Gestor de Movimientos y Gestor de Conceptos para clasificar y vincular metas con movimientos. • Interactúa con el Gestor de Balance y Reportes para mostrar indicadores de avance de metas. • Usa la interfaz IMetas implementada por el Gestor de Base de Datos para operaciones CRUD sobre las metas.
Estereotipo	«component»

Acceso al diccionario de Diagrama de Componentes:

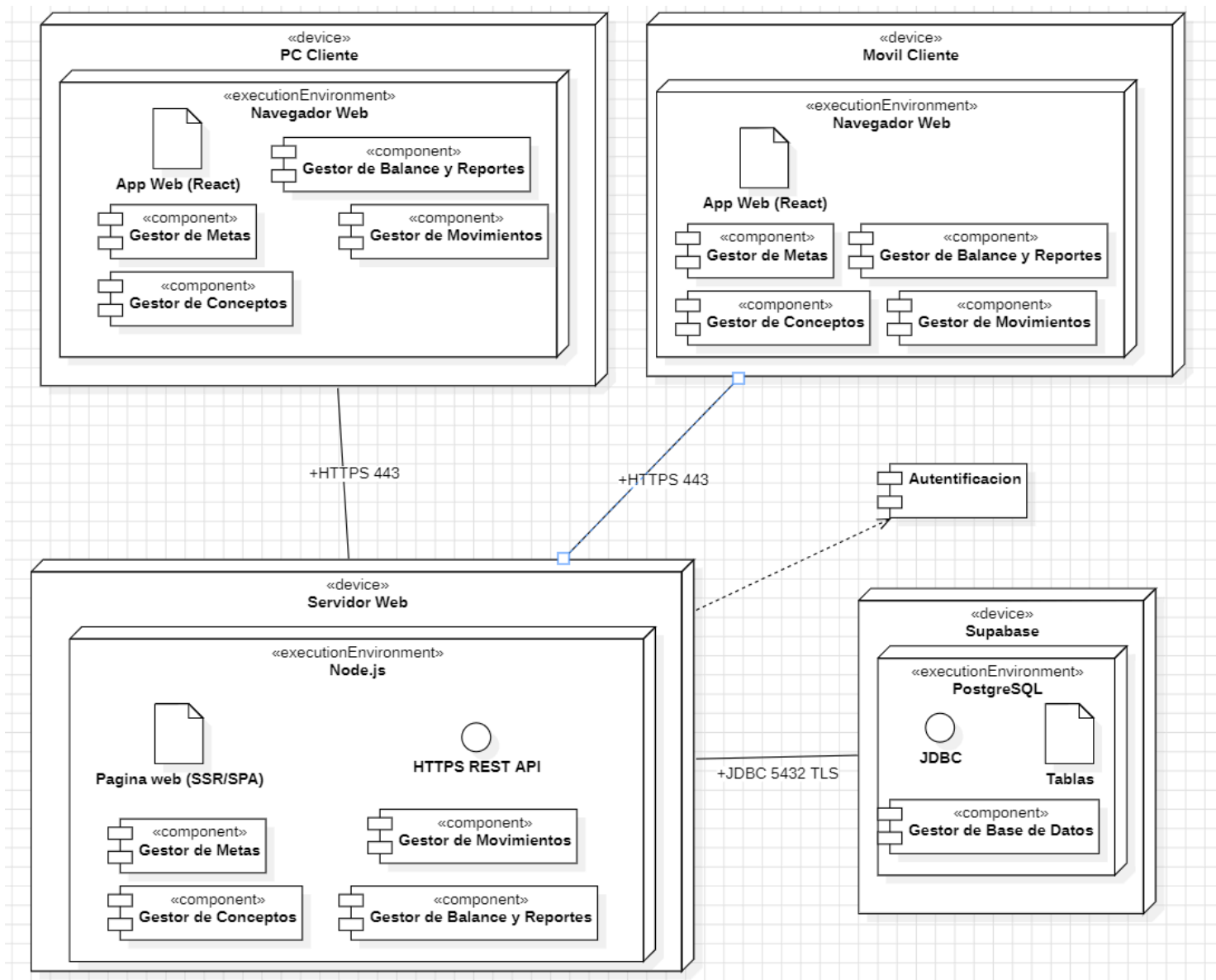
1. Para acceder al Diccionario del Diagrama del Modelo Relacional completo en un Google Docs, se debe ingresar a través del siguiente enlace proporcionado
2. Link Proporcionado:
https://docs.google.com/document/d/1AarEPDGW4Rgo8H2THCJF0MUbtI0ZvtJu_g7sv_tXD8/edit?tab=t.0

ii. Diagrama de Despliegue

En esta sección se presenta el Diagrama de Despliegue del sistema. Este diagrama describe la arquitectura física y de ejecución, especificando los nodos de hardware (dispositivos como Servidor Web o PC Cliente) y los entornos de software donde residen los artefactos. Además, modela las conexiones y protocolos de red (como HTTPS y JDBC) que comunican los distintos componentes en tiempo de ejecución.

Figura 9

Diagrama de Despliegue de Planify



1. Acceso al Diagrama de Despliegue

- a. Ingresar al repositorio de Planify en GitHub:
<https://github.com/ZeroDyna/Planify.git>
- b. En la carpeta Diagramas, descargar el DiseñoPlanify.mdj.
- c. Abrir la herramienta StarUML.
- d. En el menú principal, seleccionar File > Open. y seleccionar el archivo descargado
- e. En StarUml seleccionar la opción de Design Model.
- f. Ahí se selecciona diagrama de despliegue.

2. Diccionario del Diagrama de Despliegue

Diccionario del Diagrama de Despliegue “Planify”

Tabla 1. Diccionario del Diagrama de Despliegue - Nodo PC Cliente

Campo	Descripción
Nombre	PC Cliente
ID	DESP-01
Descripción	Dispositivo de escritorio (laptop o PC) que ejecuta un navegador que ejecuta un navegador web moderno (Chrome, Edge, Firefox) para acceder a la interfaz principal (LogIn) de Planify. Soporta pantallas grandes, entrada por teclado/mouse y sesiones prolongadas. Utiliza HTTPS 443 para comunicación segura con el Servidor Web.
Conexiones	Salida: Servidor Web (DESP-03) a través de HTTPS 443.
Requisitos	Físicos: ● Computadora de escritorio o portátil con pantalla >= 13” ● Conexión a internet estable ● Procesador: Intel i3/ AMD Ryzen 3 o superior ● RAM: por lo menos 4GB Software: ● Navegador web: Chrome, Safari, Firefox ● Sistema operativo: Windows 10/11, macOS 12+, Linux (Ubuntu 20.0
Estereotipos	<<device>>
Execution Environment	Navegador Web
Artefactos	-
Componentes asociados	● Gestor de Metas ● Gestor de Movimientos ● Gestor de Conceptos ● Gestor de Balance y Reportes

Tabla 2. Diccionario del Diagrama de Despliegue - Nodo Movil Cliente

Campo	Descripción
Nombre	Móvil Cliente
ID	DESP-02
Descripción	Dispositivo móvil (smartphone o tablet) que ejecuta un navegador web optimizado (Chrome Mobile, Safari) para acceder a Planify. Interfaz responsive con gestos táctiles. Conexión HTTPS 443 con compresión de datos.
Conexiones	● Salida: Servidor Web (DESP-03) a través de HTTPS 443.
Requisitos	Físicos: ● Smartphone o tablet con pantalla táctil. ● Conexión a internet estable. Software: ● Navegador web: Chrome Mobile, Safari iOS. ● Sistema Operativo: Android 10+, iOS 15+
Estereotipo	<<device>>
Execution Environment	Navegador Web
Artefactos	-
Componentes asociados	● Gestor de Metas ● Gestor de Movimientos ● Gestor de Conceptos ● Gestor de Balance y Reportes

Acceso al diccionario de Diagrama de Despliegue:

1. Para acceder al Diccionario del Diagrama del Modelo Relacional completo en un Google Docs, se debe ingresar a través del siguiente enlace proporcionado
2. Link Proporcionado:
https://docs.google.com/document/d/1OPIz3wbd-pi9uQdLpAoIFwsl35hTKs_pofAZKgl0nC8/edit?usp=sharing

5. Implementación

a. Aplicativo

La sección de Implementación detalla la traducción de los modelos de diseño (Gestores y Clases) a código funcional.

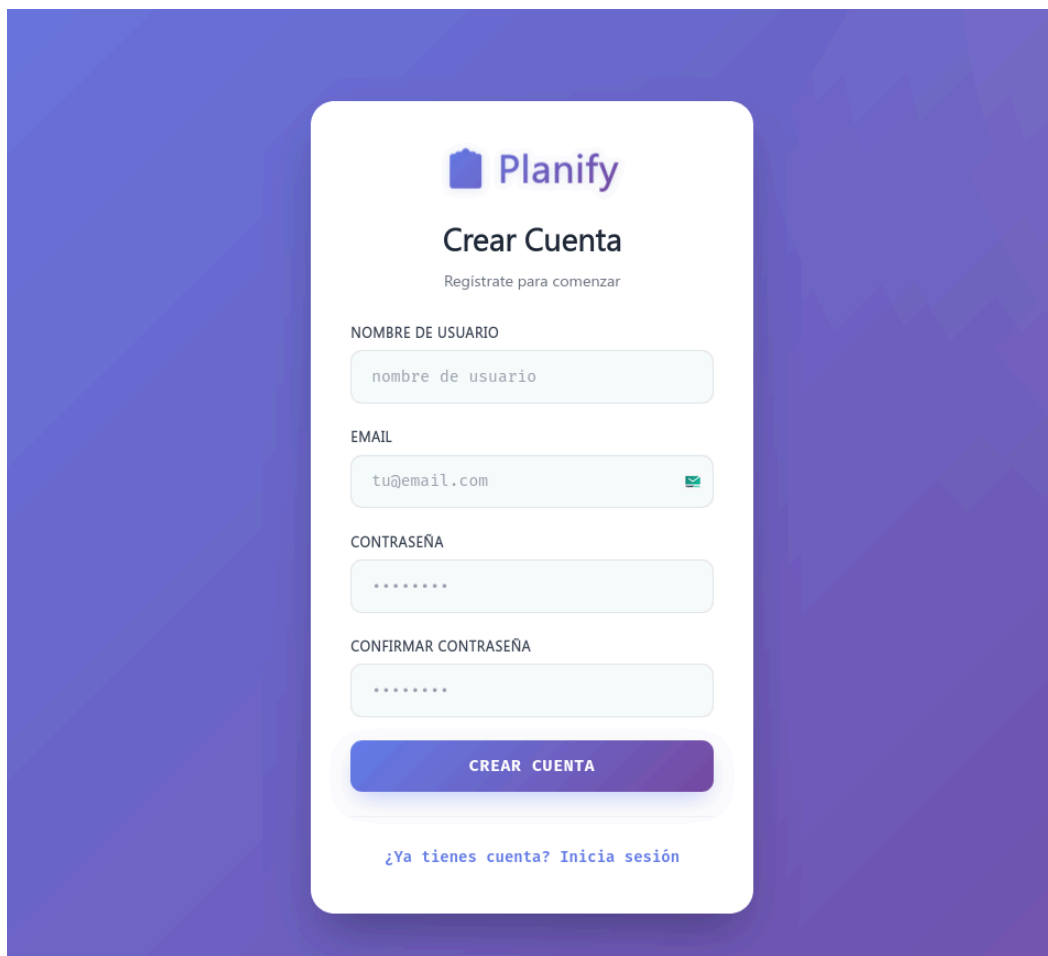
El aplicativo se implementa en un componente principal, la clase **LoginApp**, que centraliza la lógica de autenticación y maneja el estado de la interfaz de usuario (UI) a través de múltiples vistas (Mk); a continuación veremos de forma detallada los dos casos empleados

1. Registrarse (MK-002 y CU-01)

- Interfaz de registro (MK-002): Muestra los campos obligatorios: Nombre de usuario, correo, contraseña y confirmar contraseña, el componente de implementación es la función **renderLoginForm()** cuando **isSignUp** es **true**

Figura 10

Pantalla de inicio de sesión de Planify



The image shows a mobile application interface for creating a new account. The background is a solid purple color. In the center, there is a white rounded rectangle containing the registration form. At the top of the form is the Planify logo, which consists of a blue square icon with a white 'P' and the word 'Planify' in a purple sans-serif font. Below the logo is the title 'Crear Cuenta' in bold black text, followed by the subtitle 'Regístrate para comenzar' in a smaller, lighter gray font. The form contains four input fields, each with a label above it: 'NOMBRE DE USUARIO' (with placeholder text 'nombre de usuario'), 'EMAIL' (with placeholder text 'tu@email.com' and a green checkmark icon on the right), 'CONTRASEÑA' (with placeholder text '*****'), and 'CONFIRMAR CONTRASEÑA' (with placeholder text '*****'). Below these fields is a large, rounded purple button with the text 'CREAR CUENTA' in white. At the bottom of the form, there is a link that says '¿Ya tienes cuenta? Inicia sesión' in a small, light blue font.

- Gestor de registro (G-02): Encapsula la lógica de validación previa al envío y la llamada **signup**, su componente de implementación es la lógica de la función **handleauth**

Figura 10

Código de función de Gestor de Registro

```
1  try {
2    const endpoint = isSignUp ? "signup" : "token?grant_type=password";
3    const url = `${this.SUPABASE_URL}/auth/v1/${endpoint}`;
4    const body = isSignUp
5      ? { email, password, options: { data: { username } } }
6      : { email, password };
7
8    const response = await fetch(url, {
9      method: "POST",
10     headers: {
11       "Content-Type": "application/json",
12       apikey: this.SUPABASE_KEY,
13     },
14     body: JSON.stringify(body),
15   });
16
17   const data = await response.json();
```

- Validación de coincidencia: Verifica que la **Contraseña** y la de **Confirmar Contraseña** coincidan, mostrando el mensaje "**Las contraseñas no coinciden**", su componente de implementación es el Bloque **if (password !== confirmPassword)**

Figura 11

Código de función de validación de coincidencia



```

1  if (isSignUp) {
2      if (password !== confirmPassword) {
3          this.setState({
4              message: "Las contraseñas no coinciden",
5              loading: false,
6          });
7          return;
8      }

```

- Validación de longitud: Implementa la restricción de que la contraseña debe tener al menos 6 caracteres y el nombre de usuario al menos 4, su componente de implementación son los Bloques **if** (**username.trim().length < 4**) e **if (password.length < 6)**

Figura 12

Código de función de validación de longitud



```

1  // Longitud minima de usuario
2      if (username.trim().length < 4) {
3          this.setState({
4              message: "El nombre de usuario debe tener al menos 4 caracteres",
5              loading: false,
6          });
7          return;
8      }
9  // Longitud minima contraseña
10     if (password.length < 6) {
11         this.setState({
12             message: "La contraseña debe tener al menos 6 caracteres",
13             loading: false,
14         });
15         return;
16     }

```

- Correo ya registrado: El sistema maneja el error específico mostrando el mensaje: **"Este correo electrónico ya está registrado.--"**, su componente de implementación son los Bloques, su componente de implementación es **data.msg === "User already registered"**

Figura 13

Código de función de creación de mensaje de correo ya registrado

```
1 // Mensaje correo ya registrado
2 {
3   this.setState({
4     message:
5       "Este correo electrónico ya está registrado. Por favor, inicia sesión.",
6   });
7 }
```

- Registro exitoso: Confirma la creación del nuevo usuario y muestra la pantalla de notificación de verificación de correo (**renderEmailConfirmation**), su componente de implementación es Éxito en la respuesta (**response.ok**) seguido de **setState({showEmailConfirmation: true })**

Figura 14

Código de confirmación de creación de nuevo usuario

```
1 renderEmailConfirmation() {
2   return (
3     <div className="app-container">
4       <div className="login-card">
5         <div className="text-center">
6           {this.renderIcon("✉️")}
7           <h2 className="welcome-title">¡Verifica tu correo!</h2>
8           <p className="user-email" style={{ marginBottom: "1rem" }}>
9             Hemos enviado un enlace de verificación a tu correo electrónico.
10          </p>
11          <p className="subtitle" style={{ marginBottom: "1.5rem" }}>
12            Por favor, haz clic en el enlace del correo para confirmar tu
13            registro.
14          </p>
15        </div>
16      </div>
17    </div>
18  );
19 }
```

2. Recuperar contraseña (MK-017 y CU-03)

Figura 15

Pantalla de recuperación de contraseña



- Enviar el correo de recuperación (MK-004): Esta función toma el **recoveryEmail** del estado, válida que sea un correo válido, y luego utiliza la API de Supabase (**/auth/v1/recover**) para enviar el enlace de recuperación

Figura 16

Código de envío de correo de recuperación

```
1  handleForgotPassword = async () => {
2    this.setState({ loading: true, message: "" });
3    const { recoveryEmail } = this.state;
4
5    if (!recoveryEmail || !recoveryEmail.includes("@")) {
6      this.setState({
7        message: "Por favor, ingresa un correo electrónico válido.",
8        loading: false,
9      });
10     return;
11   }
12
13   try {
14     const checkEmailUrl = `${this.SUPABASE_URL}/functions/v1/check-email-exists`;
15     const checkResponse = await fetch(checkEmailUrl, {
16       method: "POST",
17       headers: {
18         "Content-Type": "application/json",
19         apikey: this.SUPABASE_KEY,
20         Authorization: `Bearer ${this.SUPABASE_KEY}`,
21       },
22       body: JSON.stringify({ email: recoveryEmail }),
23     });
24
25     const checkData = await checkResponse.json();
```

- Verificar el código (MK-011): Responsable de verificar el código OTP (**otpCode**) ingresado por el usuario con el servicio de Supabase (endpoint `/auth/v1/verify`). Si la verificación es exitosa, obtiene el **access_token** necesario para la actualización final de la contraseña y prepara el estado para mostrar el formulario de restablecimiento

Figura 17

Código de verificación de código OTP

```

1  handleVerifyOtp = async () => {
2    this.setState({ loading: true, message: "" });
3    const { recoveryEmail, otpCode } = this.state;
4
5    try {
6      const url = `${this.SUPABASE_URL}/auth/v1/verify`;
7      const response = await fetch(url, {
8        method: "POST",
9        headers: {
10         "Content-Type": "application/json",
11         apikey: this.SUPABASE_KEY,
12       },
13       body: JSON.stringify({
14         email: recoveryEmail,
15         token: otpCode,
16         type: "recovery",
17       }),
18     });
19
20     const data = await response.json();

```

- Establece nueva contraseña (MK-012): Ejecuta la actualización final de la contraseña. Primero, valida que `newPassword` y `confirmNewPassword` coincidan y cumplan con la longitud mínima de 6 caracteres. Luego, realiza una petición PUT al endpoint de usuario, utilizando el `accessToken` obtenido en el paso de verificación

3. Crear Concepto (MK-020 y CU-07)

Figura 18

Pantalla de la sección de creación de conceptos

Conceptos
Gestiona tus conceptos de ingreso y egreso

Refrescar

Cuenta
k-sketch

New Edit

Detalle del concepto

Nombre
Gustito

Tipo
Expense

Periodo
Daily

Cuenta asociada
kael.mazza@ucsp.edu.pe

Lista de Conceptos

Gustito Expense Daily	Sponsor Income Biweekly	Seguridad Expense Biweekly	Internet Expense Monthly
Cena Expense Daily	Almuerzo Expense Daily	Desayuno Expense Daily	Luz Expense Monthly
Inversion Income Weekly	Agua Expense Monthly	Salario Income Monthly	Pasivo Income Biweekly

- Crear concepto (MK-013): Responsable de crear un nuevo concepto en la base de datos Supabase con la configuración proporcionada por el usuario. Valida los campos obligatorios (nombre del concepto), envía los datos al servidor y, si la creación es exitosa, actualiza la lista de conceptos y muestra un mensaje de confirmación al usuario.

Figura 19

Código de función de creación de concepto

```

1  // FP-03: MK-013 Concepts - Crear nuevo concepto
2  // Acción: Botón New - valida y crea el nuevo concepto con su configuración
3  // Navegación: Salida a Mk-014 (Concept Created) - representado por successMessage
4  const createConcepto = async () => {
5    setMessage("");
6    setSuccessMessage("");
7    if (!cuenta) {
8      setMessage("No hay cuenta asociada a tu usuario.");
9      return;
10   }
11   // Validación: Concepto
12   if (!form.nombre_concepto.trim()) {
13     setMessage("El nombre del concepto es obligatorio.");
14     return;
15   }
16   setSaving(true);
17   try {
18     const body = {
19       correo_cuenta: cuenta.correo_cuenta,
20       nombre_concepto: form.nombre_concepto,
21       tipo: form.tipo,
22       periodo: form.periodo,
23       activo: form.activo,
24     };
25     const res = await fetch(`${SUPABASE_URL}/rest/v1/concepto`, {
26       method: "POST",
27       headers: {
28         "Content-Type": "application/json",
29         apikey: SUPABASE_KEY,
30         Authorization: accessToken ? `Bearer ${accessToken}` : "",
31         Prefer: "return=representation",
32       },
33       body: JSON.stringify(body),
34     });
35     const data = await res.json().catch(() => null);
36     if (!res.ok) {
37       console.error("Create concepto error", res.status, data);
38       setMessage(
39         data?.message || JSON.stringify(data) || "Error al crear concepto"
40       );
41       setSaving(false);
42       return;
43     }
44     await fetchConceptos();
45     if (Array.isArray(data) && data.length)
46       setSelectedId(data[0].id_concepto);
47     // Mk-014 Concept Created - Mensaje de confirmación
48     setSuccessMessage("Concepto creado correctamente");

```

4. Editar Concepto (MK-020 y CU-07)

- **Editar concepto (MK-013):** Responsable de actualizar un concepto existente en la base de datos Supabase con la nueva configuración proporcionada por el usuario. Valida que exista un concepto seleccionado y que el nombre del concepto no esté vacío, envía los datos modificados al servidor mediante una solicitud PATCH y, si la actualización es exitosa, refresca la lista de conceptos y muestra un mensaje de confirmación al usuario.

Figura 20

Código de función edición de conceptos

```

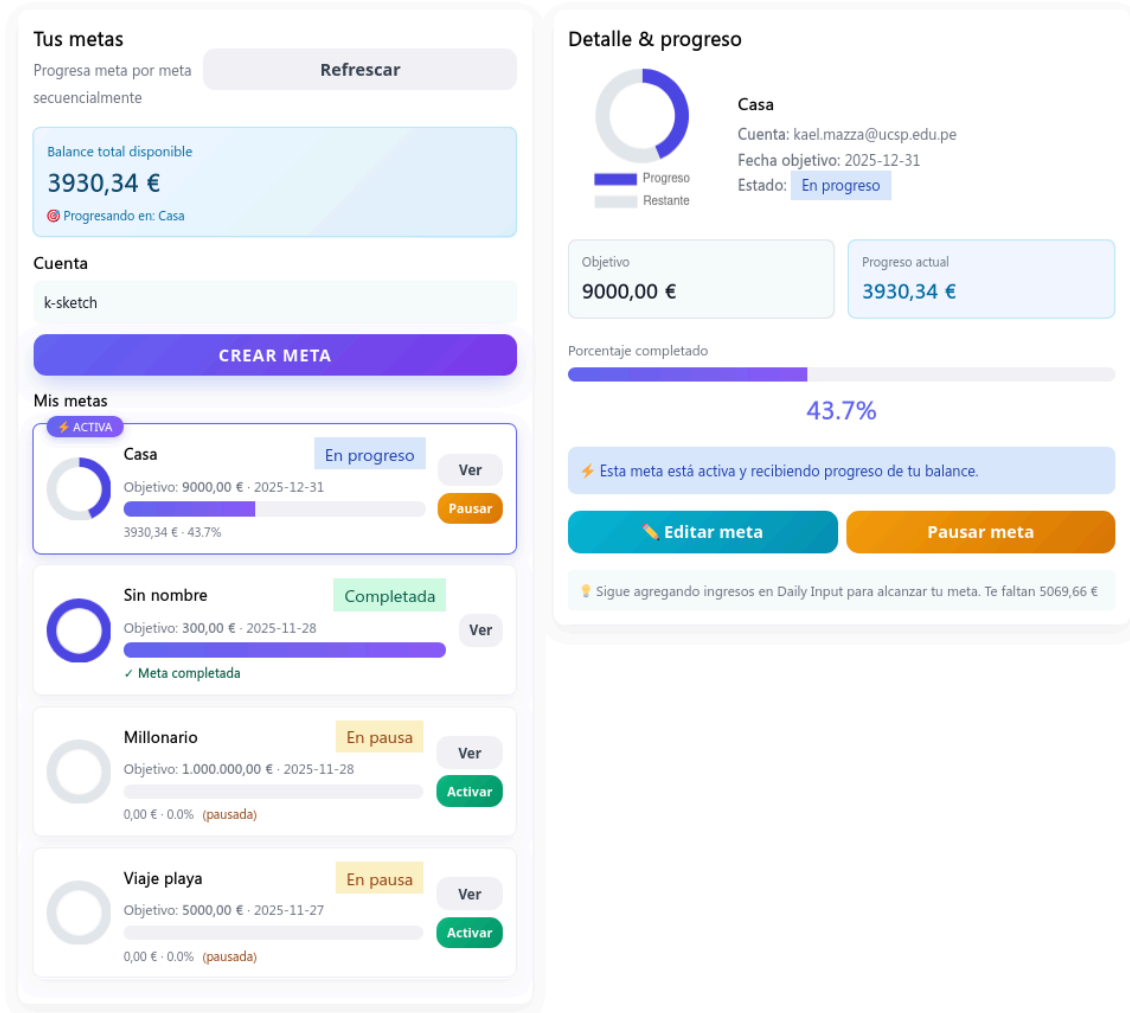
1  // FP-04: Mk-013 Concepts - Actualizar concepto existente
2  // Acción: Botón Edit - modifica un concepto con su configuración
3  const updateConcepto = async () => {
4      setMessage("");
5      setSuccessMessage("");
6      if (!cuenta || !selectedId) {
7          setMessage("Selecciona un concepto para editar.");
8          return;
9      }
10     // Validación: Concepto
11     if (!form.nombre_concepto.trim()) {
12         setMessage("El nombre del concepto es obligatorio.");
13         return;
14     }
15     setSaving(true);
16     try {
17         const body = {
18             nombre_concepto: form.nombre_concepto,
19             tipo: form.tipo,
20             periodo: form.periodo,
21             activo: form.activo,
22         };
23         const url = `${SUPABASE_URL}/rest/v1/concepto?id_concepto=eq.${selectedId}`;
24         const res = await fetch(url, {
25             method: "PATCH",
26             headers: {
27                 "Content-Type": "application/json",
28                 apikey: SUPABASE_KEY,
29                 Authorization: accessToken ? `Bearer ${accessToken}` : "",
30             },
31             body: JSON.stringify(body),
32         });
33         if (!res.ok) {
34             const data = await res.json().catch(() => null);
35             console.error("Update concepto error", res.status, data);
36             setMessage(data?.message || "Error al actualizar concepto");
37             setSaving(false);
38             return;
39         }
40         await fetchConceptos();
41         // Mk-014 Concept Created - Mensaje de confirmación para actualización
42         setSuccessMessage("Concepto actualizado correctamente");
43         setShowForm(false);
44         setIsEditing(false);

```


5. Crear Meta (MK-018 y CU-04)

Figura 21

Pantalla de la sección de Metas Planify



- Crear objetivo (Mk-018): valida y crea una meta de ahorro para la cuenta del usuario; asigna un número secuencial local (numero_objetivo) y persiste la meta en la base de datos, luego actualiza la lista en la UI

Figura 22

Código de función de creación de meta



```

1  createObjetivo = async (e) => {
2    e?.preventDefault();
3    this.setState({ message: "", successMessage: "" });
4    const { SUPABASE_URL, accessToken } = this.props;
5    const { cuenta, objetivos } = this.state;
6
7    if (!cuenta) {
8      this.setState({ message: "No hay cuenta asociada a tu usuario." });
9      return;
10   }
11
12   const { nombre, fecha_objetivo, monto_objetivo } = this.state.form;
13
14   if (!nombre || !fecha_objetivo || !monto_objetivo) {
15     this.setState({ message: "Completa todos los campos." });
16     return;
17   }
18
19   const montoNum = Number(monto_objetivo);
20   if (montoNum ≤ 0) {
21     this.setState({ message: "El monto debe ser mayor a 0." });
22     return;
23   }
24
25   this.setState({ creating: true });

```

- Establece nueva Meta (MK-008): Ejecuta la actualización final de la contraseña. Primero, valida que newPassword y confirmNewPassword coincidan y cumplan con la longitud mínima de 6 caracteres.

6. Editar Cuenta (MK-011 y CU-11)

Figura 23

Pantalla de la sección Cuenta Planify

Cuenta



Change Password

Mail
kael.mazza@ucsp.edu.pe

Full Name
k-sketch

Account name
k-sketch

Location
Location

Workplace
Workplace

Guardar cambios **Recargar**

Account email: kael.mazza@ucsp.edu.pe

- Editar cuenta (Mk-011): permite al usuario ver y modificar sus datos personales y de la cuenta (nombre, ubicación, nombre de cuenta). Guarda cambios mediante llamadas PATCH al backend y vuelve a cargar los datos para confirmar la actualización.

Figura 24

Código para guardar cambios realizados por el usuario

```

// FP-38: Guarda los cambios realizados por el usuario en sus datos personales y de
// cuenta.
// Solo permite guardar si hay datos válidos de usuario o cuenta.
handleSave = async (e) => {
  e?.preventDefault();
  this.setState({ message: "", success: "" });
  const { usuario, cuenta } = this.state;
  if (!usuario && !cuenta) {
    this.setState({ message: "No hay datos para guardar." });
    return;
  }
  this.setState({ saving: true });

  try {
    const email =
      usuario?.correo || cuenta?.correo_cuenta || this.props.user?.email;
    if (!email) {
      this.setState({
        message: "No se detectó correo para asociar cambios.",
        saving: false,
      });
      return;
    }

    // PATCH usuario -> nombre_usuario, ubicacion, lugar_trabajo
    if (usuario) {
      const payload = {
        nombre_usuario: usuario.nombre_usuario ?? "",
        ubicacion: usuario.ubicacion ?? "",
        lugar_trabajo: usuario.lugar_trabajo ?? "",
      };

```

7. Consultar Progreso Meta (MK-018 y CU-06)

Figura 25

Código de cálculo de progreso de metas

```

1  /* -----
2  * FP-32: calculateProgressForAll
3  * Calcula progreso secuencial meta por meta
4  * ----- */
5  calculateProgressForAll = (objetivos) => {
6    const { balanceTotal } = this.state;
7    let balanceAsignadoTotal = 0;
8
9    return objetivos.map((obj) => {
10      const monto_objetivo = Number(obj.monto_objetivo) || 0;
11
12      if (obj.estado === "terminada") {
13        balanceAsignadoTotal += monto_objetivo;
14        return {
15          ...obj,
16          progreso: monto_objetivo,
17          porcentaje: 100,
18        };
19      }
20
21      if (obj.estado === "en_progreso") {
22        const balanceDisponible = balanceTotal - balanceAsignadoTotal;
23        const progreso = Math.min(balanceDisponible, monto_objetivo);
24        const porcentaje =
25          monto_objetivo > 0 ? (progreso / monto_objetivo) * 100 : 0;
26
27        return {
28          ...obj,
29          progreso: Math.max(0, progreso),
30          porcentaje: Math.min(100, Math.max(0, porcentaje)),
31        };
32      }
33
34      return {
35        ...obj,
36        progreso: 0,
37        porcentaje: 0,
38      };
39    });
40  };

```

8. Registrar movimiento(MK-019 y CU-07)

Figura 26

Pantalla de la sección función de creación de meta

Daily Input

Registrar ingresos y gastos por fecha

Fecha

17 / 11 / 2025

Refrescar

Incomes

Conceptos activos

Pasivo

Salario

Inversion

Sponsor

5542,00 €

Add Income

Expenses

Conceptos activos

Aguá

Luz

Desayuno

Almuerzo

Cena

Internet

Seguridad

Gustito

Transporte

5250,00 €

Add Expense

Movimientos – 2025-11-17

Lista de movimientos registrados

Actualizar

Descripción	Tipo	Origen	Valor	
Agua	Expense	Concepto	250,00 €	Eliminar
Pasivo	Income	Concepto	12,00 €	Eliminar
Salario	Income	Concepto	1500,00 €	Eliminar
Agua	Expense	Concepto	5000,00 €	Eliminar
Salario	Income	Libre	1000,00 €	Eliminar
Suerte	Income	Libre	30,00 €	Eliminar
Prueba	Income	Libre	3000,00 €	Eliminar

Figura 27
Código para gestionar el ingreso de gastos o ganancias

```

1  handleResetPassword = async () => {
2    this.setState({ loading: true, message: "" });
3    const { newPassword, confirmNewPassword, accessToken } = this.state;
4
5    if (newPassword !== confirmNewPassword) {
6      this.setState({
7        message: "Las contraseñas no coinciden",
8        loading: false,
9      });
10     return;
11   }
12
13   if (newPassword.length < 6) {
14     this.setState({
15       message: "La contraseña debe tener al menos 6 caracteres",
16       loading: false,
17     });
18     return;
19   }
20
21   try {
22     const url = `${this.SUPABASE_URL}/auth/v1/user`;
23     const response = await fetch(url, {
24       method: "PUT",
25       headers: {
26         "Content-Type": "application/json",
27         apikey: this.SUPABASE_KEY,
28         Authorization: `Bearer ${accessToken}`,
29       },
30       body: JSON.stringify({ password: newPassword }),
31     });

```

9. Consultar Balance (MK-009 y CU-08)

Figura 28

Pantalla de la sección Balance de Planify

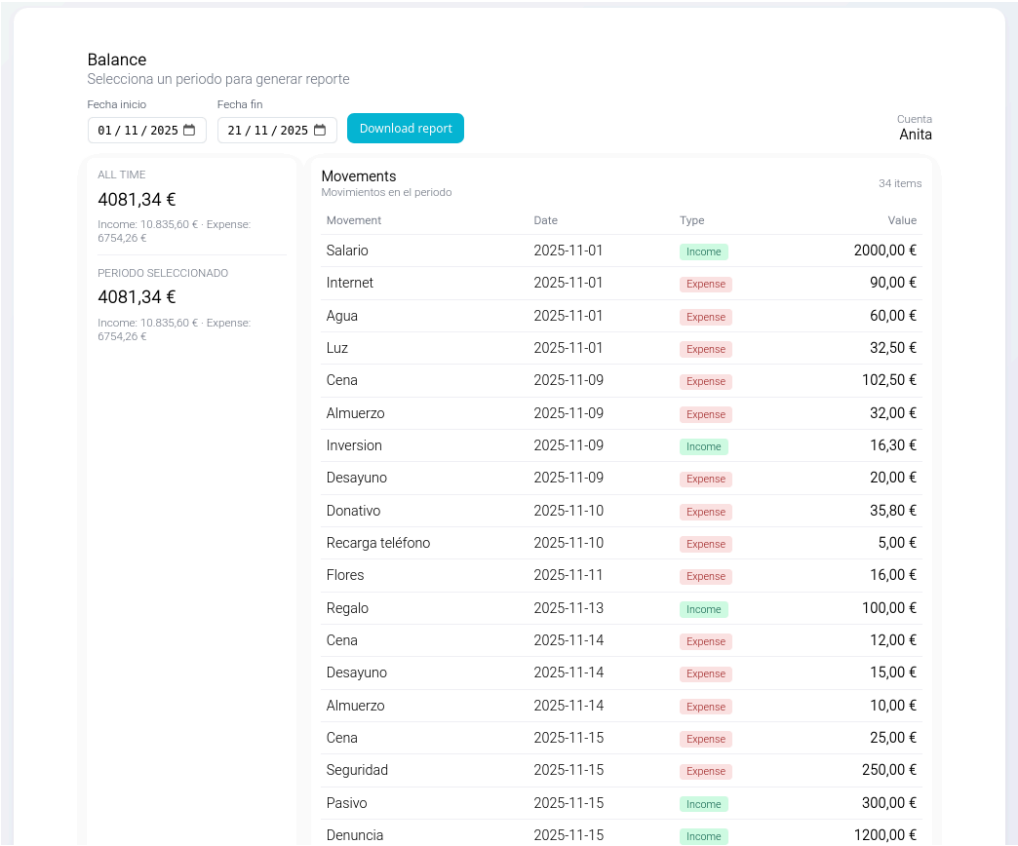


Figura 29
Códigolla de inicialización de la sección Balance

```
1  /* FP-57: initBalance
2  * Inicializa la vista: carga cuenta, movimientos del rango y totales históricos.
3  * Llamado al montar o cuando cambia user/accessToken.
4  */
5  const initBalance = async () => {
6    setLoading(true);
7    setMessage("");
8    await fetchCuentaBalance();
9    setLoading(false);
10 };
11
12 // Efectos: al montar y cuando cambia la cuenta o las fechas
13 useEffect(() => {
14   // cuando cambia user/accessToken se obtiene la cuenta
15   initBalance();
16   // eslint-disable-next-line
17 }, [user, accessToken]);
18
19 useEffect(() => {
20   // cuando cambia la cuenta o el rango, cargar movimientos
21   if (cuenta) {
22     fetchMovementsRange(startDate, endDate);
23     fetchAllActiveMovements();
24   }
25   // eslint-disable-next-line
26 }, [cuenta, startDate, endDate]);
27
28 // Formato moneda
29 const formatCurrency = (v) =>
30   new Intl.NumberFormat("es-ES", {
31     style: "currency",
32     currency: "EUR",
33     maximumFractionDigits: 2,
34   }).format(Number(v || 0));
35
36 // Totales calculados
37 const totalsRange = calculateTotals(movementsRange);
38 const totalsAll = calculateTotals(movementsAllActive);
```


10. Generar Reporte (MK-010 y CU-09)

Figura 30

Código para descargar Reporte

```
1  /* FP-56: downloadReport
2     * Valida rango, genera CSV de movementsRange y la descarga del archivo.
3     */
4  const downloadReport = async () => {
5      setMessage("");
6      const v = validateDateRange(startDate, endDate);
7      if (!v.ok) {
8          setMessage(v.reason);
9          return;
10     }
11     // Asegurar que tenemos la lista actualizada
12     await fetchMovementsRange(startDate, endDate);
13     setDownloading(true);
14     try {
15         // Construir CSV: encabezado + filas
16         const header = [
17             "fecha",
18             "nombre_movimiento",
19             "tipo",
20             "monto",
21             "correo_cuenta",
22             "origen",
23         ];
24         const rows = (movementsRange || []).map((m) => [
25             m.fecha,
26             m.nombre_movimiento?.replace(/"/g, "'") || "",
27             m.tipo ? "Income" : "Expense",
28             m.monto ?? "",
29             m.correo_cuenta || "",
30             m.sourceTable === "concepto" ? "Concepto" : "Libre",
31         ]);
32         const csv = [header, ...rows]
33             .map((r) => r.map((c) => `"${c}"`).join(","))
34             .join("\n");
35         const blob = new Blob([csv], { type: "text/csv;charset=utf-8;" });
36         const url = URL.createObjectURL(blob);
37         const a = document.createElement("a");
38         a.href = url;
39         const fileName = `planify_report_${startDate}_to_${endDate}.csv`;
40         a.download = fileName;
41         document.body.appendChild(a);
42         a.click();
43         a.remove();
44         URL.revokeObjectURL(url);
45         setMessage("Download successful");
46     } catch (err) {
47         console.error("downloadReport error", err);
48         setMessage("Error generando el reporte.");
49     } finally {
50         setDownloading(false);
51     }
52 }
```

11. Iniciar sesión (MK-001 y CU-02)

- Editar cuenta (Mk-007): permite al usuario ver y modificar sus datos personales y de la cuenta (nombre, ubicación, nombre de cuenta). Guarda cambios mediante llamadas PATCH al backend y vuelve a cargar los datos para confirmar la actualización.

Figura 31

Código para guardar cambios realizados por el usuario en sus datos personales

```
// FP-38: Guarda los cambios realizados por el usuario en sus datos personales y de
// cuenta.
// Solo permite guardar si hay datos válidos de usuario o cuenta.
handleSave = async (e) => {
  e?.preventDefault();
  this.setState({ message: "", success: "" });
  const { usuario, cuenta } = this.state;
  if (!usuario && !cuenta) {
    this.setState({ message: "No hay datos para guardar." });
    return;
  }
  this.setState({ saving: true });

  try {
    const email =
      usuario?.correo || cuenta?.correo_cuenta || this.props.user?.email;
    if (!email) {
      this.setState({
        message: "No se detectó correo para asociar cambios.",
        saving: false,
      });
      return;
    }
  }

  // PATCH usuario -> nombre_usuario, ubicacion, lugar_trabajo
  if (usuario) {
    const payload = {
      nombre_usuario: usuario.nombre_usuario ?? "",
      ubicacion: usuario.ubicacion ?? "",
      lugar_trabajo: usuario.lugar_trabajo ?? "",
    };
  }
}
```

Acceso al Código Fuente del Aplicativo

1. Ingresar al repositorio de Planify en GitHub:
<https://github.com/ZeroDyna/Planify>
2. Ingresar a la carpeta Implementacion/src
3. Descargar el archivo App.js
4. Acceder al archivo para la visualización del código fuente

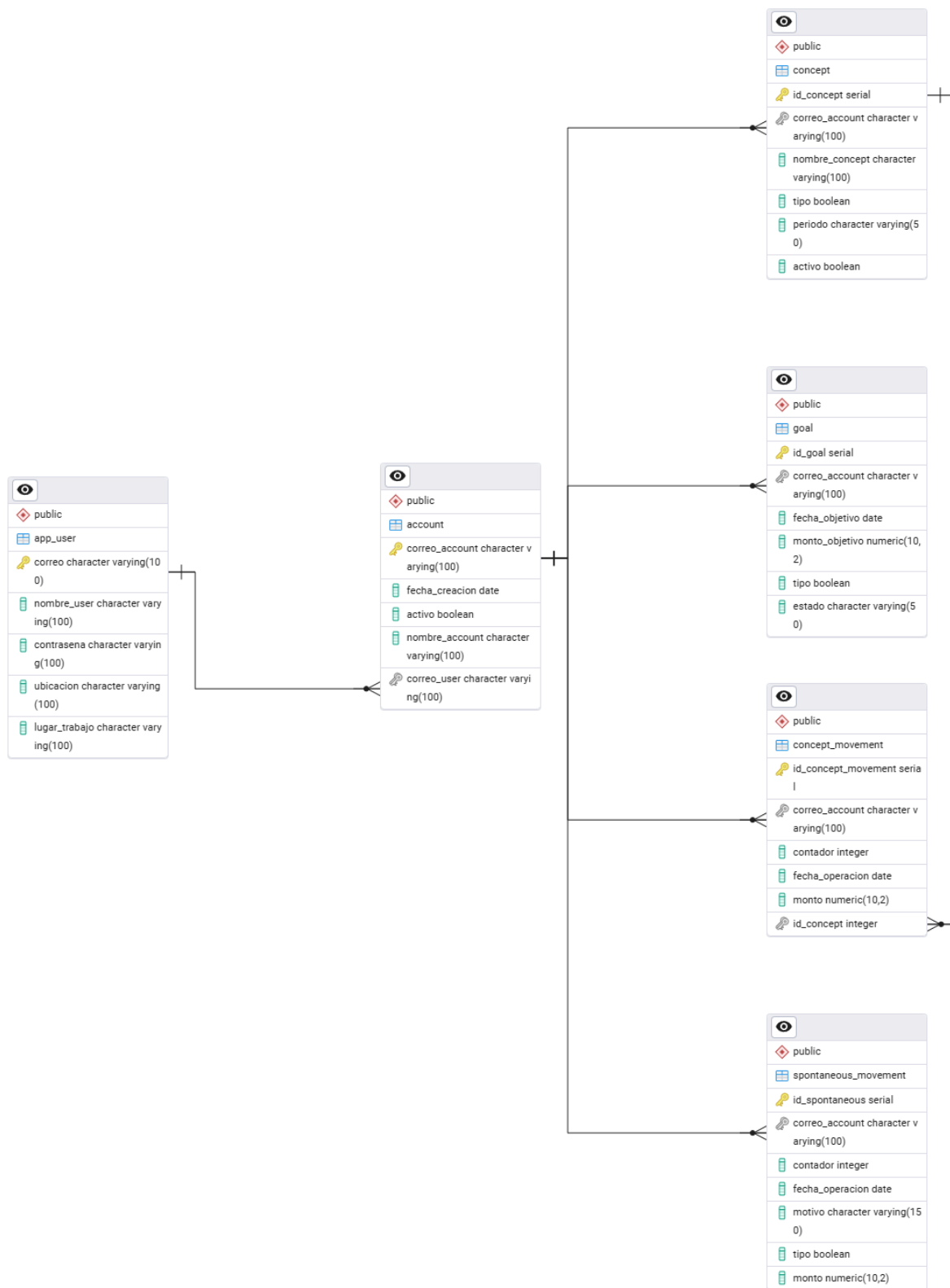
b. Modelo Físico

La sección del Modelo Físico describe la **traducción e implementación** del Modelo Relacional en una estructura de base de datos operativa y persistente.

Esta implementación se realiza **con lo planificado en el diseño**

Figura 32

Modelo Físico (véase Figura 7 en la siguiente página)



Acceso al Modelo Físico

1. Para acceder al modelo físico ya implementado hacer click en el link proporcionado

2. Link Proporcionado:
<https://supabase.com/dashboard/project/sopeknspwpauugvarnvw>
3. Iniciar sesión en una cuenta autorizada, si no está registrado en Supabase debe crear una cuenta.

1. Diccionario del Modelo Físico

El diccionario del Diagrama del Modelo Físico, detalla la estructura lógica y los componentes de nuestra base de datos.

Tabla 1. app_user

Campo	Tipo de Dato	Restricciones	Descripción
correo	VARCHAR(100)	PRIMARY KEY, NOT NULL	Identificador único de usuario
nombre_usuario	VARCHAR(100)	NOT NULL	Nombre del usuario registrado.
contrasena	VARCHAR(100)	NOT NULL	Contraseña cifrada para autenticación.
ubicacion	VARCHAR(100)	—	Ciudad o región del usuario.
lugar_trabajo	VARCHAR(100)	—	Lugar o institución donde trabaja el usuario.

Descripción: Contiene la información básica y de autenticación de los usuarios del sistema

Tabla 2. account

Campo	Tipo de Dato	Restricciones	Descripción
correo_cuenta	VARCHAR(100)	PRIMARY KEY, NOT NULL	Identificador único de la cuenta del usuario.
fecha_creacion	DATE	NOT NULL	Fecha en que se creó la cuenta.

Tabla 2. Continuación desde la página anterior

activo	BOOLEAN	DEFAULT TRUE	Indica si la cuenta se
--------	---------	--------------	------------------------

			encuentra activa.
nombre_cuenta	VARCHAR(100)	NOT NULL	Nombre descriptivo de la cuenta.
correo_usuario	VARCHAR(100)	FOREIGN KEY → app_user(correo)	Usuario propietario de la cuenta.

Descripción: Permite gestionar las cuentas financieras asociadas a cada usuario.

Acceso al Diccionario del Modelo Físico

1. Para acceder al diccionario completo hacer click en el link proporcionado
2. Link Proporcionado:
<https://docs.google.com/document/d/14NGGDV1xQfL9WsiJ90DzOONIP1xkLrGWj-qjbzIpbjM/edit?tab=t.0>

Pruebas

6. Pruebas

La fase de pruebas del sistema **Planify** se desarrolló conforme a la estrategia y lineamientos establecidos en el Plan de Pruebas del Software Planify.

Dicho plan define los objetivos, alcance, tipos de prueba, recursos y metodología utilizada para validar el correcto funcionamiento del sistema. El enfoque principal se centró en asegurar la integridad de los datos y el cumplimiento de los requisitos funcionales en los módulos de registro, inicio de sesión, gestión de tareas y perfil de usuario.

Durante la ejecución se realizaron **pruebas unitarias, de integración, de sistema y de validación**, siguiendo casos de prueba formales que cubren los escenarios principales y alternos del sistema. Estas pruebas permitieron verificar la interacción entre los distintos componentes, la respuesta ante datos inválidos, la generación correcta de reportes financieros y la consistencia entre el frontend y el backend.

A modo ilustrativo, la siguiente tabla presenta uno de los diseños de caso de prueba ejecutados durante esta fase:

TABLA XXXVIII
Prueba de Validación III

Nombre	Validación - verificación de registro de un nuevo movimiento por concepto.
ID	CD-VA-09
Tipo de Prueba	Caja Negra
Objetivo	Verificar que el sistema registre correctamente los movimientos ingresados por el usuario al intentar registrar movimientos en la sección de daily input, asegurando que se registre con el monto y concepto correcto.
Especificación de Entradas	Escenario principal: 1. En el módulo daily input se selecciona el botón “add income”. 2. Seleccionar la casilla “usar concepto existente”. 3. Seleccionar un concepto. 4. Ingresar un monto válido. 5. Presionar el botón “guardar”. Escenario alterno: 4. Ingresar un monto inválido. 5. Presionar el botón “guardar”.
Requisitos de Procedimiento	El usuario debe haberse registrado en el sistema previamente. El usuario debe haber registrado al menos un concepto en el sistema previamente. El usuario debe encontrarse en la sección de daily input.
Especificación de Salidas	1. Si el monto ingresado es válido: ● El sistema registra el movimiento en el sistema. ● Se muestra el mensaje: “<i>Movimiento guardado correctamente</i>”. 2. Si las credenciales son incorrectas: El sistema no registra el movimiento en el sistema. El sistema muestra el mensaje: “<i>Incorrect value: monto inválido</i>”.

Acceso al Plan de Pruebas del Software Planify

- Para acceder al Plan de Pruebas del Software Planify hacer click en el link proporcionado: [☰ Plan de Pruebas del Software Planify](#)

Despliegue

7. Despliegue Planify

El despliegue contempla la preparación del entorno local, la configuración necesaria y la puesta en funcionamiento de la aplicación para fines de validación interna previa a su liberación.

a. Manual de Usuario

Son videos breves que muestran el uso de cada funcionalidad de los módulos del aplicativo paso a paso.

Este es el listado de tutoriales que se muestran en el manual de usuario:

Autenticación & cuenta

- Tutorial_de_registro
- Tutorial_Inicio_de_sesión
- Tutorial_Cambio_contraseña
- Tutorial_Ver_perfil
- Tutorial_editar_datos

Daily Input

- Tutorial_Registrar_Movimiento_por_Concepto
- Tutorial_Registrar_Movimiento_Espontaneo
- Tutorial_Filtrar_movimientos_por_fecha
- Tutorial_Editar_movimiento

Conceptos (categorías)

- Tutorial_Crear_concepto
- Tutorial_detalles_concepto
- Tutorial_editar_concepto

Objetivos / Metas

- Tutorial_Crear_meta
- Tutorial_editar_meta
- Tutorial_Activar_pausar_meta
- Tutorial_Ver_progreso_y_graficos

Balance / Reportes

- Tutorial_Consultar_datos
- Tutorial_Exportar_Reporte
- Tutorial_Analizar_balance

Acceso al Manual de Usuario

1. Para acceder a los videos del Manual de Usuario de Planify hacer click en el link proporcionado.
2. Link proporcionado:
https://drive.google.com/drive/folders/18oJoRtVIYFOI85QwFt2SLAcK_TOs13KK?usp=share_link

b. Plan de Entrega

Este Plan de Entrega describe formalmente el producto software final desarrollado, así como su alcance funcional, los componentes incluidos, los requisitos técnicos, el procedimiento de instalación y los responsables de la entrega.

Acceso al Plan de entrega

1. Ingresar al repositorio de Planify en GitHub:
<https://github.com/ZeroDyna/Planify>
2. Ingresar a la carpeta Planes

8. Mantenimiento

El mantenimiento comprende la organización y ejecución de las acciones técnicas previstas para asegurar la continuidad y correcta operatividad del sistema durante su ciclo de uso.

a. Plan de Mantenimiento

El mantenimiento propuesto abarca acciones **correctivas** y **preventivas**, garantizando que el software se mantenga funcional, actualizado y alineado con los estándares de calidad establecidos.

Acceso al Plan de Mantenimiento

1. Ingresar al repositorio de Planify en GitHub:
<https://github.com/ZeroDyna/Planify>
2. Ingresar a la carpeta *Planes*
3. Ingresar al archivo llamado *Plan de Pruebas*

9. Referencias

Calderón Ruiz, G. (s.f). *Proyecto* de Laboratorio.

Gurock Software GmbH. (2023). TestRail (Version 9.3) [Test Management Software].

<https://www.testrail.com>

MkLab. (2023). StarUML (Version 6.3.1) [Computer Software]. <https://staruml.io/>

SeleniumHQ. (2024). Selenium WebDriver (Version 4.15.0) [Software Testing Framework]. <https://www.selenium.dev>